

VanniKawachh

Complete code, algorithm, formula and hardware reference

Distributed AI acoustic intelligence + autonomous drone response

This document is generated directly from the source tree. Every algorithm, formula, constant and library below is traced to the exact file (and line) that implements it, together with the alternatives that were considered and the reason the current choice was made.

Project	VanniKawachh - acoustic distress detection network with autonomous UAV response
Repository	github.com/SV-1411/drone (public)
Languages	Python 3.11 (hub, flight stack, ML), C/C++ Arduino (ESP32 firmware), JavaScript/React (dashboard)
Code size	~10,700 lines across 60 source files
Test status	80 unit tests pass, 0 failures (re-run and verified this session); end-to-end SITL flight validated

HOW TO USE THIS DOCUMENT

Sections 1-5 are the story you tell. Section 6 is every algorithm and formula with its location. Sections 7-9 are the comparison and cheat-sheet tables to memorise. Section 12 is 50 rehearsed panel questions. Section 13 is what to say when the panel finds a weakness - answering those honestly scores higher than bluffing. If you only have one hour, read Sections 2, 3, 6-summary, 9 and 12.

The five sentences that must be automatic

1. Solar-powered ESP32-S3 nodes on street poles listen continuously and run a tiny neural network on-device (Stage 1) to spot a scream or a call for help.
2. A hit sends a 25-byte AES-128-encrypted, HMAC-authenticated alert over LoRa (no internet, 5-10 km) plus a 4-second audio clip over WiFi.
3. A Raspberry Pi 5 hub re-verifies the clip with a large pretrained audio model (Stage 2 / PANNs), fuses it with motion, darkness and time-of-day evidence, and only then decides.
4. If two independent thresholds are both crossed, the hub POSTs the node's surveyed GPS coordinates to the drone stack, which flies a fully autonomous mission: arm, take off, navigate, hover and record evidence, drop a first-aid kit, return to launch.
5. The claimable novelty is not 'send a drone to a GPS point' - it is the two-stage verification plus the safety-interlock layer: every flight-mode command is confirmed by autopilot telemetry before the mission proceeds.

Contents

1 System architecture - the five layers

The diagram to draw on the board, the layer-to-code map, and why two stages at all

2 Which code goes into which hardware (the burn map)

What is flashed where, the full node pin map, and the trap question about the Pixhawk

3 Complete file map

All 60 source files: what each one does, which machine runs it, key function names

4 One incident, end to end

The 18-step walkthrough to rehearse, each step tied to a file and line

5 Every algorithm and formula, with its location

MFCC, the classifier, quantisation, FFT, Adam, PANNs, fusion, crypto, haversine, routing, verified mode transition, failsafe arbitration - each with formula, location, reason and alternatives

6 Master comparison table (landscape)

31 design decisions: what we use, where, the alternatives, and why this one

7 Every library (landscape)

Version, side, where used, role, and the alternative for each of 29 dependencies

8 Numbers cheat sheet

Every constant in the system with its file and the reason it has that value

9 Dataset, training protocol and what to claim about accuracy

Classes, data sources, augmentation, the two leakage checks reviewers look for

10 What is proven, what is not

The evidence table, and the twelve gaps to name yourself before the panel does

11 Fifty rehearsed panel questions

Concept, architecture, algorithms, security, flight safety, testing and roadmap

12 Live demo runbook and closing notes

What to demo ranked by risk, what to say first, what never to say

IF YOU HAVE ONLY ONE HOUR

Read Section 2 (the burn map) and Section 8 (the numbers) properly - those are pure recall and the panel will test them. Skim Section 5 for the formulas you are shakiest on. Then read Section 11 out loud. Section 10 is your insurance: knowing your own gaps better than the panel does turns every hard question into a prepared answer.

1 System architecture - the five layers

Draw this on the board if asked. Data flows strictly left to right; each layer can be tested independently, and each layer degrades gracefully if the next one is missing.

DATA FLOW

```

L1 SENSING NODE      ESP32-S3 + INMP441 mic + PIR + LDR + SX1278 LoRa
    | 25-byte sealed alert (LoRa 433 MHz, SF9) [no internet]
    | 4 s WAV clip (WiFi HTTP POST)
    v
L2 HUB                Raspberry Pi 5 + gateway ESP32 (LoRa RX -> USB serial)
    | lora_gateway -> pipeline -> verifier -> fusion -> dispatcher
    | POST /trigger {lat, lon, incident_type, priority, deliver_kit}
    v
L3 TRIGGER API        FastAPI + priority queue + SQLite (uvicorn)
    | in-process call: MissionExecutor.run_mission(spec)
    v
L4 FLIGHT CORE        13-state machine, failsafe arbiter, MAVLink client
    | MAVLink over TCP (SITL) or UART (real Pixhawk)
    v
L5 AIRCRAFT           Pixhawk 2.4.8 running ArduPilot Copter + SG90 + Pi camera

DASHBOARD             React + Leaflet <-- HTTP / WebSocket --> L3
  
```

Layer-to-code map

Layer	Hardware	Code that runs there	Language	Key libraries
L1 Sensing node	ESP32-S3 dev board, INMP441 I2S mic, HC-SR501 PIR, LDR, SX1278 LoRa, 18650 + TP4056 + solar	firmware/node/node.ino, stage1.cpp, stage1.h, stage1_nn.h	C/C++ (Arduino)	driver/i2s.h, LoRa (Sandeep Mistry), WiFi, HTTPClient, Preferences (NVS), mbedtls (AES + HMAC), optional TFLM
L2a LoRa gateway	Any ESP32 dev board + SX1278, on the Pi's USB	firmware/gateway/gateway.ino	C/C++ (Arduino)	SPI, LoRa
L2b Hub	Raspberry Pi 5 (or any PC for the demo)	hub/ package: main, lora_gateway, packets, node_registry, verifier, fusion, pipeline, dispatcher, webapp, sim_drone	Python 3.11	pycryptodome (AES), hmac/hashlib (stdlib), numpy, pyserial, requests, FastAPI, uvicorn, panns-inference + torch (optional)
L3 Trigger API	Companion computer on the drone (Pi Zero 2 W) or a ground PC	trigger_api/: main, models, mission_queue, store	Python 3.11	FastAPI, pydantic v2, uvicorn, websockets, sqlite3 (stdlib)
L4 Flight core	Same companion computer	flight_core/: mission_executor, mavlink_interface, failsafe_handler, obstacle_avoidance, payload_release, camera_recorder, config	Python 3.11	dronekit 2.9.2, pymavlink, picamera2 (on the Pi only)
L5 Aircraft	Pixhawk 2.4.8 flight controller, M8N GPS + compass, 4S LiPo, SG90 servo, Pi Camera 3	ArduPilot Copter firmware - NOT our code. We only send it MAVLink messages.	C++ (ArduPilot)	ArduPilot / ArduCopter >= 4.3 (copter-3.3 in SITL)
Training / offline	Dev laptop or GPU box	ml/: mfcc, train_stage1, train_stage1_numpy, train_gpu, infer_nn, eval_pipeline, make_bootstrap_dataset, record_samples	Python 3.11	numpy (always), tensorflow + librosa + scikit-learn (optional CNN path)

Layer	Hardware	Code that runs there	Language	Key libraries
Dashboard	Any browser	dashboard/src/: App.jsx, Map.jsx, Telemetry.jsx, IncidentLog.jsx	JavaScript (React 18)	React 18, Vite 5, Leaflet 1.9, react-leaflet 4

Why a two-stage design at all (the question that always comes)

A single big model cannot live on a pole: PANNs/CNN14 is ~80 MB of float weights and needs hundreds of MB of RAM. An ESP32-S3 has ~512 KB of SRAM. So Stage 1 is a deliberately tiny, high-recall, low-precision detector that runs on the node and only decides 'worth a look'. Stage 2 is the heavy, high-precision model on the Pi that decides 'actually distress'. This is a cascade / coarse-to-fine classifier: cheap filter first, expensive confirmation second.

- Bandwidth: only ~0.2% of audio is ever transmitted, so a solar node and a LoRa link are sufficient.
- Privacy: no continuous audio leaves the pole - only 4 s around a detection.
- False alarms: a false dispatch is expensive and erodes trust, so precision is bought back at Stage 2 where compute is cheap.
- Power: continuous inference at ~0.5 Hz on 2 s windows fits a solar/18650 budget; continuous streaming would not.

2 Which code goes into which hardware (the burn map)

THE TRAP QUESTION

"What code do you flash onto the Pixhawk?" - None. The Pixhawk runs stock ArduPilot Copter firmware, flashed once with Mission Planner. Our Python code is an external MAVLink client that commands it. Only the two ESP32 boards receive code we wrote; the Pi 5, the Pi Zero and the laptop have software installed, not burned.

Hardware	What we put on it	How it gets there	What that code does
ESP32-S3 (sensing node)	firmware/node/node.ino + stage1.cpp + stage1.h (+ stage1_nn.h from ml/out/ after training, or model_data.cc for the TFLM build)	Arduino IDE / PlatformIO -> USB flash. Board: 'ESP32-S3 Dev Module', PSRAM = OPI enabled.	I2S audio capture at 16 kHz, 4 s ring buffer in PSRAM, MFCC every ~0.5 s on the newest 2 s, Stage-1 classification, PIR + LDR read, AES-128-CTR + HMAC packet sealing, LoRa TX, WiFi clip upload, alert counter persisted in NVS
ESP32 (LoRa gateway)	firmware/gateway/gateway.ino	Arduino IDE -> USB flash. Any classic ESP32 dev module.	Deliberately dumb bridge: receives LoRa frames and prints one line 'RX <hex> RSSI <dbm>' on USB serial at 115200. Holds no keys, does no verification - so stealing the gateway gains an attacker nothing.
Raspberry Pi 5 (hub)	The hub/ Python package + requirements-hub.txt (pycryptodome, pyserial, numpy, requests, FastAPI, uvicorn; optionally panns-inference + torch)	git clone + pip install. Run: python -m hub.main --serial /dev/ttyUSB0	Reads the gateway serial stream, unseals/authenticates packets, replay-checks the counter, looks up the node's surveyed coordinates, waits for the WiFi clip, runs Stage-2 verification, fuses evidence into a severity, applies two thresholds, POSTs /trigger, and serves the dashboard
Pi Zero 2 W (companion computer, on the drone)	flight_core/ + trigger_api/ + requirements.txt (dronekit, pymavlink, FastAPI, uvicorn, pydantic); picamera2 for the camera	git clone + pip install, run under systemd; MAVLINK_CONNECTION=/dev/serial0,921600	Receives /trigger, queues the mission by priority, runs the 13-state flight state machine, sends MAVLink commands to the Pixhawk, runs the failsafe monitor at 1 Hz, records evidence video, fires the payload servo, streams telemetry over WebSocket
Pixhawk 2.4.8 (flight controller)	NOTHING OF OURS. Stock ArduPilot Copter (>= 4.3) firmware + a parameter set (docs/HARDWARE_INTEGRATION.md section 5).	Flashed once via Mission Planner / QGroundControl over USB.	Runs the actual attitude/position control loops, EKF sensor fusion, its own hardware failsafes and geofence. Our code only sends it MAVLink: SET_MODE, ARM, TAKEOFF, goto, DO_SET_SERVO, and reads its HEARTBEAT/telemetry back.
Dev laptop / PC	The whole repo: SITL simulator (dronekit-sitl copter-3.3), dashboard (Vite dev server), ml/ training scripts, pytest suite	pip install -r requirements.txt; npm install in dashboard/	Simulates the aircraft (SITL), trains and exports the Stage-1 model, serves the React dashboard, runs the 80-test unit suite and the end-to-end flight test
A phone (demo-only path)	Nothing installed - it just opens a web page the hub serves: /node (sensing node) and /drone-phone (drone unit)	Browser on the same WiFi, over HTTPS so mic + GPS permissions work	Lets you demo the whole detection-to-dispatch chain with no hardware at all: the phone records real audio, uploads it to /phone-alert, the hub runs Stage 1 + Stage 2 + fusion, and an animated drone flies on the dashboard map

Node pin map - memorise this table

Peripheral	Signal	ESP32-S3 GPIO	Set in code at
INMP441 mic (I2S)	WS (word select / LRCLK)	GPIO 4	firmware/node/node.ino:43
INMP441 mic (I2S)	SCK (bit clock / BCLK)	GPIO 5	firmware/node/node.ino:43
INMP441 mic (I2S)	SD (serial data in)	GPIO 6	firmware/node/node.ino:43
INMP441 mic	L/R -> GND (selects left channel), VDD 3V3	-	I2S_CHANNEL_FMT_ONLY_LEFT, node.ino:73
SX1278 LoRa (SPI)	SCK	GPIO 12	node.ino:44 / SPI.begin(12,13,11,NSS)

Peripheral	Signal	ESP32-S3 GPIO	Set in code at
SX1278 LoRa (SPI)	MISO	GPIO 13	node.ino:188
SX1278 LoRa (SPI)	MOSI	GPIO 11	node.ino:188
SX1278 LoRa	NSS (chip select)	GPIO 10	node.ino:44
SX1278 LoRa	RST	GPIO 9	node.ino:44
SX1278 LoRa	DIO0 (RX-done IRQ)	GPIO 8	node.ino:44
HC-SR501 PIR	OUT (digital)	GPIO 7	node.ino:42
LDR + 10 kOhm divider	ADC in	GPIO 1	node.ino:42, analogRead >> 4

Gateway ESP32 (classic VSPI): NSS = GPIO 5, SCK = GPIO 18, MOSI = GPIO 23, MISO = GPIO 19, RST = GPIO 14, DIO0 = GPIO 2 - firmware/gateway/gateway.ino:17. Both radios are set to 433 MHz with spreading factor 9; node and gateway MUST match or nothing is received.

KNOWN DOC/CODE MISMATCH - KNOW IT BEFORE THEY FIND IT

docs/HARDWARE_INTEGRATION.md section A2 lists the mic as WS = GPIO 5 and SCK = GPIO 4, which is swapped relative to the firmware (node.ino:43 sets WS = 4, SCK = 5). The firmware is authoritative - wire to the firmware. The same doc paragraph also says '1 s windows' where the code uses a 2 s window. Both are documentation bugs, not code bugs; say so plainly if asked and note that the firmware constant is the single source of truth.

Never-forget hardware rules

- Never power the SX1278 and transmit without the 433 MHz antenna attached - the power amplifier will burn out.
- INMP441 is 3.3 V only. The ESP32-S3 is 3.3 V logic throughout.
- The Pixhawk servo/AUX rail is NOT powered internally - the SG90 needs a 5 V BEC on the AUX rail (docs/HARDWARE_INTEGRATION.md section 13).
- SITL_MODE=1 relaxes ArduPilot pre-arm checks (ARMING_CHECK=0). It must stay UNSET on real hardware - flight_core/mission_executor.py:_relax_sitl_arwing_checks explicitly refuses to touch parameters unless that env var is 1.
- An RC transmitter with a mode switch is mandatory on every real flight even though the mission is autonomous - it is the human kill path.
- The node's alert counter lives in NVS (Preferences), not RAM. If it reset on reboot, the hub would reject every packet after the first boot as a replay.

3 Complete file map - what every file is for

60 source files, ~10,700 lines. If the panel points at any file, this table is the answer. 'Runs on' tells you which machine executes it.

firmware/ - the only code that is actually burned

File	Lines	Runs on	Purpose / key symbols
firmware/node/node.ino	223	ESP32-S3 node	Main sketch: i2sInit(), readFrame(), 4 s PSRAM ring buffer, newestWindow(), deriveNodeKey(), buildPacket() (25-byte sealed packet), sendLoraAlert(), uploadClip() (builds a 44-byte WAV header by hand), setup()/loop(). Trigger rule at line 213: class != background AND confidence >= 0.60 AND 15 s since last alert.
firmware/node/stage1.h	41	ESP32-S3 node	Stage-1 interface + the three compile-time backends. Declares Stage1Result {cls, confidence}, stage1_init(), stage1_infer(), stage1_event_code().
firmware/node/stage1.cpp	219	ESP32-S3 node	buildTables() (Hamming, mel filterbank, DCT basis), fft() (in-place iterative radix-2), computeMFCC() (mirrors ml/mfcc.py step for step), then three backends: USE_TFLM_STAGE1 (int8 CNN in TFLM, 40 KB arena), USE_NN_STAGE1 (26-24-4 MLP, pure float matmuls), or the default energy heuristic.
firmware/node/stage1_nn.h	13	ESP32-S3 node	Auto-generated by ml/train_stage1_numpy.py: s1nn_mu, s1nn_sd (26 standardisation values), s1nn_W1 (624), s1nn_b1 (24), s1nn_W2 (96), s1nn_b2 (4). This is the trained model, as C float arrays - about 3 KB.
firmware/gateway/gateway.ino	41	Gateway ESP32	setup() opens LoRa at 433 MHz SF9; loop() prints 'RX <hex> RSSI <dbm>'. No crypto, no state.

hub/ - Stage-2 hub service (Raspberry Pi 5)

File	Lines	Purpose / key symbols
hub/config.py	105	HubConfig frozen dataclass, from_env(). Thresholds, master key, serial port, the four drone stations (DEFAULT_DRONE_BASES), cruise speed. Env is read at construction, so variables must be set before the process starts.
hub/packets.py	110	The wire format. Alert dataclass, node_key() (HMAC key derivation), _ctr_cipher(), seal(), unseal(), PacketError. 25-byte packet: 9 B header + 8 B ciphertext + 8 B MAC.
hub/node_registry.py	80	Node {node_id, lat, lon, name, last_counter}; NodeRegistry loads/saves nodes.json atomically (tmp + os.replace) under a threading.Lock, and bump_counter() persists the replay high-water mark.
hub/lora_gateway.py	71	SerialGateway (pyserial, parses 'RX <hex> RSSI') and SimGateway (in-memory queue for tests and the Phase-0 demo). Both expose the same packets() generator - that is why the pipeline is testable with no radio.
hub/verifier.py	117	Stage2Verifier auto-selects PannsBackend (AudioSet tagging, sums the distress-relevant class probabilities) or falls back to EnergyHeuristicBackend (RMS + spectral centroid + burstiness). load_wav_mono() uses only the stdlib wave module.
hub/fusion.py	48	fuse(alert, audio_score) -> Severity {score, priority, reasons}. The weighted evidence sum, plus the priority escalation rule.
hub/pipeline.py	148	AlertPipeline.process_packet() - the whole decision chain in one readable function; process_clip() for the phone path. Incident dataclass is the record written to the dashboard log.
hub/dispatcher.py	46	Dispatcher.dispatch() POSTs /trigger with X-API-Key, handles 429 (queue full) distinctly from other failures, returns the mission id.
hub/webapp.py	634	FastAPI app: POST /clip/{node}/{ctr} (node clip upload), POST /phone-alert (phone path with stage1_phone() gate), /drone_state, /drones, /drone-mission, /drone-report, /incidents, /nodes, and three server-rendered HTML pages: / (dashboard), /node, /drone-phone.
hub/sim_drone.py	277	SimDrone (animated mission for the no-hardware demo), DroneFleet._nearest() (the nearest-station selection - the one place distance ranking happens), FleetDispatcher, SimDispatcher, PhoneDrone (a second phone reporting real GPS).

File	Lines	Purpose / key symbols
hub/main.py	137	Entry point. --sim injects one test alert, --serial COM3 reads the real gateway, --web-only + --https run the phone-test mode. Starts the web app in a daemon thread and generates a self-signed cert via openssl so phones will grant mic/GPS.
hub/nodes.json	-	The surveyed node registry - node_id -> lat/lon/name/last_counter. Currently one demo node.

flight_core/ - the flight state machine (unchanged from v1)

File	Lines	Purpose / key symbols
flight_core/mission_executor.py	780	The heart. MissionState (13 states), MissionSpec, Waypoint, TelemetrySnapshot, MissionExecutor: run_mission(), _wait_until_safe_to_start(), _arm_and_takeoff(), _set_mode_confirmed() (line 504 - the patented idea), _raw_set_mode(), _goto_avoiding(), _goto_waypoint() (stall detector), _hover(), _deliver_kit(), _rtl_and_wait_landed(), _abort(), _safe_rtl(), _wait_for_disarm().
flight_core/failsafe_handler.py	163	FailsafeHandler: a 1 Hz background thread checking link age, battery, GPS, geofence and mission timeout; _emit() implements the arbitration lattice (LAND outranks RTL, never downgraded, one event per name unless escalating).
flight_core/mavlink_interface.py	83	The Python 3.10+ collections ABC shim that must run BEFORE 'import dronekit'; connect_vehicle() with retry/backoff; haversine_distance_m(); relative_location(); wait_for_gps_lock().
flight_core/obstacle_avoidance.py	150	Deterministic map-based avoidance: Obstacle, _to_local/_to_global (equirectangular projection), _closest() (point-to-segment), path_clear(), plan_route() (recursive detour insertion, depth cap 6), load_obstacles_from_env().
flight_core/payload_release.py	47	set_servo() sends MAV_CMD_DO_SET_SERVO through pymavlink; release_kit() opens, waits 2 s, re-closes. A failed release never blocks the mission.
flight_core/camera_recorder.py	70	CameraRecorder.start()/stop() using picamera2 + H264Encoder at 4 Mbit/s. If picamera2 is absent (any dev machine or SITL) every call is a logged no-op.
flight_core/config.py	135	Config frozen dataclass with ~35 env-overridable fields: altitudes, speeds, thresholds, failsafe limits, servo PWM values, API settings. Read at construction, not at import.

trigger_api/, ml/, dashboard/, tests/, scripts/

File	Lines	Purpose
trigger_api/main.py	244	FastAPI surface: POST /trigger, GET /mission/{id}, POST /mission/{id}/cancel, POST /mission/{id}/waypoint, GET /missions, /missions/archive, /telemetry, /health, WS /ws/telemetry. Geofence rejection at the edge, X-API-Key guard on writes, lifespan hook that eager-connects off the event loop and commands RTL on shutdown.
trigger_api/models.py	80	Pydantic v2 models. Bounds are enforced here so the flight core never sees an impossible value: lat +/-90, lon +/-180, altitude 2-120 m, hover 0-3600 s, priority in {low, normal, high, critical}.
trigger_api/mission_queue.py	195	MissionQueue: single-drone serial execution, priority-ordered deque, depth cap 20 (HTTP 429 beyond it), history pruning that never drops pending/running, cancel() semantics, worker thread.
trigger_api/store.py	120	SQLite persistence. On boot, anything left 'queued'/running' by a crash is marked 'interrupted'. Persistence failures are logged and swallowed - they must never take down the dispatch path.
ml/mfcc.py	86	THE single definition of the feature front-end, mirrored by firmware/node/stage1.cpp. Deliberately hand-written instead of librosa so the C port is exact.
ml/train_stage1_numpy.py	170	The trainer that actually produced the committed model: NumPy-only, MFCC -> mean+std pooling -> 26-24-4 MLP, Adam, exports stage1_nn.h and .npz.
ml/train_stage1.py	167	TensorFlow path: 4-layer CNN over the (123 x 13) MFCC image, int8 post-training quantisation, exports .tflite plus a C byte array for TFLM.
ml/train_gpu.py	287	The honest-metrics trainer: streams files one at a time (~1 GB RAM), auto-downloads ESC-50, optional Kaggle scream set, real-noise-mix augmentation at 0-20 dB SNR, splits by FILE to prevent leakage, reports precision/recall/F1/confusion matrix.
ml/infer_nn.py	46	Reference Python implementation of exactly what the C firmware does - so the deployed path can be unit-tested.

File	Lines	Purpose
ml/eval_pipeline.py	129	Evaluates the two-stage cascade end to end: Stage-1 per-class recall, background false-trigger rate, Stage-2 score separation, and how many Stage-1 triggers the hub confirms.
ml/make_bootstrap_dataset.py	218	Synthesises a small labelled dataset so the pipeline can be trained and demonstrated before field recordings exist.
ml/record_samples.py	78	Records your own labelled clips into ml/data/<class>/.
dashboard/src/App.jsx + Map.jsx + Telemetry.jsx + IncidentLog.jsx	~416	React 18 + Leaflet viewer: live map with breadcrumb path, telemetry panel, incident log, dispatch + cancel buttons, API-token field, follow toggle. No flight controls beyond dispatch/cancel by design.
tests/test_units.py	517	39 tests: failsafe arbitration, the verified mode setter against a ModeRejectingVehicle mock, abort interlock, queue, store, API validation, config.
tests/test_hub.py	219	14 tests: packet round-trip, tamper, wrong key, replay, bad length/magic, registry persistence, fusion, verifier ordering, pipeline gating (dispatch / no-dispatch / unknown node / no clip).
tests/test_mfcc.py + test_stage1_nn.py	113	7 tests: MFCC shape, determinism, pad/crop, tone vs silence separation, finiteness; and that the deployed NN path separates classes and returns confidences in [0,1].
tests/test_obstacle_avoidance.py	83	7 pure-geometry tests: projection round-trip, direct path when clear, blocking detection, single and double detour clearance, env parsing.
tests/test_phone_mode.py	126	5 tests for the no-hardware demo path: silence does not dispatch, a scream does, a new alert moves the drone, pages render, drone-phone reporting.
tests/test_full_mission.py	332	The end-to-end SITL flight test - spawns dronekit-sitl + uvicorn, triggers a real mission, asserts 8 required checks. ~5-6 minutes, prints PASS/FAIL.
scripts/demo_phase0.py	180	The zero-hardware full-chain demo: synthesises a scream WAV, seals it into a packet, runs the hub pipeline, dispatches into SITL, watches the flight to completion.

4 One incident, end to end - the walkthrough to rehearse

If you can narrate these 18 steps with the file names, you can answer almost any 'how does it work' question. Timings are design targets, not measured field numbers.

1. Node: capture

i2s_read() pulls 512-sample frames (32 ms) at 16 kHz from the INMP441. The mic is 24-bit-in-32; readFrame() shifts right by 14 to get int16. node.ino:79

2. Node: ring buffer

Every sample is written into a 4-second circular buffer in PSRAM (clipBuf, 64000 samples = 128 KB). This is why the clip that gets uploaded contains the audio from BEFORE the trigger. node.ino:57, 202

3. Node: window

Every 16 frames (~0.5 s) newestWindow() copies the newest 2 s (32000 samples) out of the ring into a linear buffer. node.ino:101

4. Node: MFCC

computeMFCC() turns those 32000 samples into a 123 x 13 feature matrix: pre-emphasis, Hamming window, 512-point radix-2 FFT, 40 mel filters, log, DCT-II, keep 13. stage1.cpp:90

5. Node: Stage 1

The classifier gives (class, confidence). Default build = energy heuristic; USE_NN_STAGE1 = the 26-24-4 MLP; USE_TFLM_STAGE1 = the int8 CNN under TensorFlow Lite Micro. stage1.cpp:114-219

6. Node: trigger gate

Fire only if class != background AND confidence >= 0.60 AND at least 15 s since the last alert (refractory period, so one incident is one alert). node.ino:213

7. Node: context

Read PIR (digitalRead, GPIO 7) and LDR (analogRead on GPIO 1, 12-bit value shifted right 4 to fit a byte). node.ino:216

8. Node: seal

buildPacket() increments the NVS-backed counter, derives the per-node key, AES-128-CTR-encrypts the 8-byte payload, appends the first 8 bytes of HMAC-SHA256 over header+ciphertext. 25 bytes total. node.ino:120

9. Node: transmit

LoRa.beginPacket / write / endPacket at 433 MHz, SF9. Then uploadClip() builds a WAV header in memory and POSTs the whole 4 s ring buffer (oldest-first) to the hub over WiFi. node.ino:144, 155

10. Gateway

The gateway ESP32 receives the frame and prints 'RX <50 hex chars> RSSI -87' on USB serial. It never decrypts anything. gateway.ino:33

11. Hub: parse

SerialGateway.packets() yields the raw 25 bytes. lora_gateway.py:56

12. Hub: authenticate

unseal() checks length, magic 'VK', version, recomputes the MAC with hmac.compare_digest (constant-time), rejects counter <= last_counter (replay), then decrypts. Any failure = packet dropped, no drone. packets.py:90

13. Hub: locate

The packet carries only a node id, so the registry supplies the surveyed lat/lon. That is why a 25-byte packet is enough and why no node needs a GPS module. node_registry.py:get()

14. Hub: Stage 2

_wait_for_clip() polls hub/clips/<node>_<ctr>.wav for up to 8 s. If the clip arrives, PANNs (or the energy fallback) scores it 0..1. If it never arrives, the score degrades to stage1_confidence x 0.6 - the system still works, just more conservatively. pipeline.py:56, 88

15. Hub: fuse

fuse() combines audio score (0.60), Stage-1 confidence (0.15), PIR (0.10), darkness (0.08) and night-hours (0.07) into a severity, and sets priority high if severity >= 0.75 or (audio >= 0.6 and PIR). fusion.py:35

16. Hub: decide

Two independent gates must both pass: `audio_score >= VERIFY_THRESHOLD (0.50) AND severity >= DISPATCH_THRESHOLD (0.60)`. Otherwise the incident is logged with no dispatch. `pipeline.py:104`

17. Hub: dispatch

POST `/trigger {lat, lon, priority, incident_type:'acoustic_distress', deliver_kit:true}` with the X-API-Key header. 429 (queue full) is logged distinctly. `dispatcher.py:22`

18. Drone

The API validates bounds and geofence, enqueues by priority, and the worker thread runs the mission: connect -> GPS lock -> relax checks (SITL only) -> GUIDED confirmed -> arm confirmed -> takeoff to 95% of target altitude -> goto (routing around keep-out zones) -> hover while recording evidence -> descend to 3 m, fire the servo, climb back -> RTL -> confirmed landed and disarmed. `mission_executor.py:332`

THE SENTENCE THAT ANSWERS 'WHY IS THIS SAFE?'

Three mechanisms, all in `flight_core`. (1) Verified mode transition: no mode change is ever assumed - it is re-sent every 700 ms as raw MAVLink until the autopilot's own HEARTBEAT reports the new mode. (2) Landing interlock: every abort path blocks until the aircraft has landed AND disarmed, so the queue can never start a flight against an airborne vehicle. (3) Failsafe arbitration: LAND outranks RTL and is never downgraded, GPS loss is debounced over 3 consecutive bad samples, and each failsafe fires once per mission unless it escalates.

5 Every algorithm and formula, with its location

For each: what it computes, the formula, where it lives, why it is there, and what the alternative would have been. This is the section the panel will dig into.

5.1 MFCC feature extraction - the audio front end

Where ml/mfcc.py (Python, training) and firmware/node/stage1.cpp:90 (C, on-device). The two MUST agree bit-for-bit in behaviour.

Why here A raw 2-second clip is 32000 numbers. MFCCs compress it to $123 \times 13 = 1599$ numbers that describe the shape of the sound spectrum the way human hearing does - which is exactly what distinguishes a scream from traffic.

MFCC PIPELINE

Constants: $SR=16000$ Hz $window=2.0$ s (32000 samples) $n_fft=512$ $hop=256$
 $n_mels=40$ (0-8000 Hz) $n_mfcc=13$ $pre-emphasis=0.97$ $eps=1e-6$
 $frames = 1 + (32000 - 512) / 256 = 123$ output shape (123, 13)

1. Pre-emphasis (a 1st-order high-pass; boosts the quiet high frequencies)

$$x'[n] = x[n] - 0.97 * x[n-1]$$
2. Framing + Hamming window (512 samples = 32 ms, hop 256 = 50% overlap)

$$w[n] = 0.54 - 0.46 * \cos(2\pi * n / (N-1)), \quad N = 512$$

$$frame_i[n] = x'[i*256 + n] * w[n]$$
3. Power spectrum via FFT (257 bins = $n_fft/2 + 1$)

$$P[k] = |FFT(frame)[k]|^2$$
4. Mel filterbank - 40 triangular filters, equally spaced on the mel scale

$$mel(f) = 2595 * \log_{10}(1 + f/700) \quad \text{Hz} \rightarrow \text{mel}$$

$$f(mel) = 700 * (10^{(mel/2595)} - 1) \quad \text{mel} \rightarrow \text{Hz}$$

$$bin(m) = \text{floor}((n_fft + 1) * f(mel_m) / SR)$$

$$\text{rising edge: } H_m[k] = (k - 1) / (c - 1) \quad \text{for } 1 \leq k < c$$

$$\text{falling edge: } H_m[k] = (r - k) / (r - c) \quad \text{for } c \leq k < r$$
5. Log mel energies (log makes loudness additive and compresses dynamic range)

$$E[m] = \log(\sum_k H_m[k] * P[k] + 1e-6)$$
6. DCT-II -> decorrelate, keep the 13 lowest coefficients

$$c[i] = \sum_{m=0}^{39} E[m] * \cos(\pi * i * (2m + 1) / (2 * 40))$$

Alternatives and why not

- librosa.feature.mfcc - rejected on purpose. Its exact filterbank and DCT normalisation are awkward to reproduce in C, and if the ESP32's features differ even slightly from the training features, the model silently degrades. Writing the front end by hand made the C mirror exact and testable (tests/test_mfcc.py).
- Raw log-mel spectrogram (no DCT) - what most modern CNNs use, and arguably better for a CNN. Kept the DCT because it drops 40 numbers per frame to 13, which matters for both the MLP input size and the microcontroller's RAM.
- Learned front end (SincNet, wav2vec) - far better accuracy, impossible on an ESP32-S3 and unnecessary when Stage 2 does the heavy lifting.
- Edge Impulse - would generate front end + model with guaranteed feature parity. A legitimate alternative; rejected to keep the pipeline self-contained, inspectable and free of a vendor dependency (noted in stage1.cpp's header).

5.2 Stage-1 classifier - the model that runs on the pole

Three interchangeable backends selected at compile time. Only one is committed as trained weights today: the MLP.

Build flag	Model	Size	Needs	Status
(default, none)	Energy + high-band heuristic: $0.6 \cdot \min(1, \text{rms}/0.08) + 0.4 \cdot \min(1, \text{band})$	~0 KB	nothing	Bring-up fallback so the chain runs before any model exists
-DUSE_NN_STAGE1	MFCC -> mean+std pooling (26) -> Dense 24 ReLU -> Dense 4 softmax	~3 KB of floats	no ML library at all	TRAINED AND COMMITTED (ml/out/stage1_nn.npz, firmware/node/stage1_nn.h)
-DUSE_TFLM_STAGE1	4-layer int8 CNN over the (123 x 13) MFCC image	~33 KB tflite + 40 KB arena	TensorFlow Lite Micro (Chirale_TensorFlowLite or esp-tflite-micro)	Code path written and reviewed; not yet trained/flashed on hardware

DEPLOYED STAGE-1 FORWARD PASS

MLP path (what is actually deployed) - ml/train_stage1_numpy.py + infer_nn.py
+ firmware/node/stage1.cpp:162

Feature pooling over the 123 frames, per MFCC coefficient c:

```
mean[c] = (1/F) * sum_f M[f][c]
std[c]  = sqrt( (1/F) * sum_f (M[f][c] - mean[c])^2 )    (population std, ddof=0)
x = concat(mean, std)                                -> 26 features
```

Standardisation with the training-set statistics baked into the header:

```
z[i] = (x[i] - mu[i]) / sd[i]
```

Forward pass:

```
h = ReLU(z @ W1 + b1)    W1 is 26x24, b1 is 24    ReLU(u)=max(0,u)
logits = h @ W2 + b2     W2 is 24x4, b2 is 4
p[k] = exp(logits[k] - max(logits)) / sum_j exp(logits[j] - max(logits))
class = argmax(p), confidence = max(p)
```

Total parameters: $26 \cdot 24 + 24 + 24 \cdot 4 + 4 = 748$ (plus 52 normalisation constants)

The 'max(logits)' subtraction in the softmax is not cosmetic - it is the standard numerical-stability trick. Without it exp() overflows on large logits. Note it is present in all three implementations (Python trainer, Python reference, C firmware) which is how you know they agree.

Why mean+std pooling instead of feeding the whole 123 x 13 matrix?

- A Dense layer on 1599 inputs would need ~38,000 weights - 50x the model, and it would overfit badly on a small dataset.
- Mean+std is a deliberate bag-of-frames summary: it captures 'what does this sound like on average, and how much does it fluctuate'. Burstiness is exactly what separates a scream from steady traffic noise, and std captures it.
- The cost is that it throws away temporal ORDER - it cannot tell 'help' from 'pleh'. That is a genuine limitation and the reason the CNN path exists: a CNN over the time-frequency image keeps the ordering. Say this if challenged; it is the correct answer.

The TFLM CNN path (ml/train_stage1.py, ml/train_gpu.py)

CNN ARCHITECTURES

train_stage1.py (compact, for the microcontroller):

```
Input (123, 13, 1)
Conv2D(8, (4,3), stride (2,1), ReLU)      # stride in time = cheap downsampling
Conv2D(16, (4,3), stride (2,1), ReLU)
Conv2D(24, (4,3), stride (2,2), ReLU)
GlobalAveragePooling2D()                  # time-invariant, no huge flatten
Dense(24, ReLU) -> Dropout(0.3) -> Dense(4, softmax)
```

train_gpu.py (stronger, for honest metrics):

```
Conv2D(16,(5,3),same,ReLU) -> BatchNorm -> MaxPool(2,1)
Conv2D(32,(5,3),same,ReLU) -> BatchNorm -> MaxPool(2,2)
Conv2D(48,(3,3),same,ReLU) -> GlobalAveragePooling
Dense(48,ReLU) -> Dropout(0.35) -> Dense(4, softmax)
```

GlobalAveragePooling instead of Flatten is a deliberate choice: it makes the model insensitive to WHERE in the 2 s window the scream happened, and it keeps the parameter count tiny. Dropout is only in the dense head, where overfitting lives.

5.3 int8 post-training quantisation (the TFLM path)

QUANTISATION

Quantise a float feature into the model's int8 input tensor:

```
q = clamp( round(x / S) + Z, -128, +127 )
```

Dequantise the int8 output back to a probability:

```
p = (q - Z) * S
```

S = tensor scale, Z = zero point - both read from the tflite model at runtime

```
firmware/node/stage1.cpp:148 (in->params.scale, in->params.zero_point)
```

The scales are calibrated by running ~200 real training samples through the float model - the 'representative dataset' - ml/train_stage1.py:export_tflite_int8()

- Why int8: 4x smaller weights, and integer arithmetic is far faster on a microcontroller with no FPU-heavy vector units. ~33 KB model instead of ~130 KB.
- Alternative - float32 TFLM: simpler, no calibration, but 4x the flash and slower. Alternative - quantisation-aware training: better accuracy retention, more training complexity; the right next step if int8 costs measurable accuracy.

5.4 Radix-2 iterative FFT on the microcontroller

Where firmware/node/stage1.cpp:65 - fft(re, im, 512)

FFT

Two phases, textbook Cooley-Tukey:

1. Bit-reversal permutation of the 512 inputs (in place, no scratch buffer)
2. $\log_2(512) = 9$ butterfly stages; for stage length L the twiddle factor is
 $W = \exp(-2\pi i / L)$ computed incrementally by complex multiply
 butterfly: $u = a, \quad v = b * W$
 $a' = u + v, \quad b' = u - v$

Cost: $O(N \log N) = 512 * 9 \sim 4600$ butterflies per frame, 123 frames per window.

Naive DFT would be $O(N^2) = 262144$ per frame - about 57x slower. That difference is exactly why real-time inference on a 240 MHz microcontroller is possible.

- Python side uses `numpy.fft.rfft` (which is FFTW-class and returns only the 257 non-redundant bins). The C side computes the full complex 512-point transform and uses bins 0..256 - mathematically identical for real input, because the spectrum of a real signal is conjugate-symmetric.
- Alternative: the ESP-DSP library's assembly-optimised FFT, which would be measurably faster. Hand-written was chosen so the file has no external dependency and can be read line-by-line against ml/mfcc.py.

5.5 Training: cross-entropy loss + Adam optimiser (hand-implemented)

Where ml/train_stage1_numpy.py:105-125 - the entire optimiser is ~8 lines of NumPy.

LOSS, BACKPROP, ADAM

Loss - categorical cross-entropy on softmax outputs:

$$L = -(1/N) * \sum_n \log(p[n, y_n])$$

The gradient of (softmax + cross-entropy) collapses to a single subtraction:

$$dL/d\text{logits} = (p - Y) / N \quad Y = \text{one-hot labels}$$

This is why the code has no explicit softmax derivative - that cancellation is the reason softmax and cross-entropy are always paired.

Backprop through the hidden layer:

```
gw2 = h^T @ dlogits          gb2 = sum(dlogits)
dh  = (dlogits @ W2^T) * (h_pre > 0)    <- ReLU derivative is a 0/1 mask
gw1 = X^T @ dh              gb1 = sum(dh)
```

Adam update, per parameter, at step t:

```
m = 0.9 * m + 0.1 * g          (1st moment - momentum)
v = 0.999 * v + 0.001 * g^2    (2nd moment - variance)
m_hat = m / (1 - 0.9^t)        (bias correction; m starts at 0)
v_hat = v / (1 - 0.999^t)
p = p - lr * m_hat / (sqrt(v_hat) + 1e-8)
```

Hyperparameters: lr = 3e-3, beta1 = 0.9, beta2 = 0.999, eps = 1e-8, 400 epochs, full-batch gradient descent, He initialisation: $W \sim N(0, \sqrt{2/n_{in}})$

- Why He initialisation ($\sqrt{2/n_{in}}$) and not Xavier: He is the correct variance scaling for ReLU networks, because ReLU zeroes half the activations.
- Why Adam and not plain SGD: it adapts a per-parameter step size, so it converges without hand-tuning a learning-rate schedule - which matters when the whole trainer must be 170 lines of NumPy.
- Why NumPy and not TensorFlow: TensorFlow is a ~500 MB download that stalls on some networks. This trainer needs only NumPy and the standard library, so the model can be retrained anywhere, including on the Pi. The TensorFlow path still exists for the stronger CNN.

5.6 Stage-2 verification - PANNS (primary) and the energy heuristic (fallback)

Where hub/verifier.py - Stage2Verifier picks a backend at construction; if torch/panns-inference import fails it logs a warning and falls back.

STAGE 2 - PANNS

PANNS backend (hub/verifier.py:92) - Pretrained Audio Neural Networks, CNN14, trained on Google AudioSet (2 million clips, 527 sound classes).

```
clipwise_probs = AudioTagging.inference(audio)    # 527 probabilities
distress_idx   = every class whose label contains one of
                {scream, shout, yell, crying, wail, groan, whimper, screaming}
score = min(1.0, sum(clipwise_probs[i] for i in distress_idx))
```

Audio is resampled to 32 kHz mono (PANNS' training rate) by load_wav_mono().

STAGE 2 - ENERGY HEURISTIC

Energy-heuristic backend (hub/verifier.py:60) - the dev/demo fallback.

Three hand-designed acoustic cues, each squashed into [0,1], then a weighted sum:

```

1. Loudness      rms = sqrt(mean(x^2));  loudness = min(1, rms / 0.15)
2. Spectral      centroid = sum(|X[k]| * f[k]) / sum(|X[k]|)      (Hz)
   centroid      highness = clamp((centroid - 400) / 1600, 0, 1)
   (screams sit high in the spectrum; traffic rumble sits low)
3. Burstiness    envelope = |x| averaged in 20 ms blocks
   burst = min(1, (max(env) / mean(env)) / 8)
   (a scream is a short peak against a quiet floor; a steady machine is not)

score = 0.45 * loudness + 0.35 * highness + 0.20 * burst

```

SAY THIS BEFORE THEY ASK IT

The energy heuristic is explicitly NOT an accuracy claim - the docstring in hub/verifier.py says so, and it is labelled 'energy-heuristic (dev fallback)' in the logs and on the dashboard. It exists so the entire chain can be demonstrated on any laptop without a 2 GB torch install. On the Pi 5, PANNs is the real Stage 2. Volunteering this distinction is worth more marks than being caught on it.

- Alternatives for Stage 2: YAMNet (TF-Hub MobileNet on AudioSet, lighter, slightly weaker), AST / Audio Spectrogram Transformer (state of the art, heavier), or fine-tuning PANNs on your own scream data (the correct future step - transfer learning would beat both).
- Why sum the distress classes rather than take a max: several AudioSet labels (Screaming, Shout, Yell, Crying) describe the same event and split the probability mass between them. Summing recombines it; min(1.0, ...) keeps the result a score.

5.7 Multi-sensor evidence fusion -> severity and priority

Where hub/fusion.py:35

FUSION

```

darkness = 1 - (light / 255)                # 1.0 = pitch dark (LDR byte)
is_night = 1 if (hour >= 20 or hour < 6) else 0

severity = 0.60 * audio_score                # Stage-2 verification dominates
          + 0.15 * stage1_confidence         # the node's own opinion
          + 0.10 * pir                      # was something moving?
          + 0.08 * darkness                 # unlit spot
          + 0.07 * is_night                 # unsafe hour
severity = clamp(round(severity, 3), 0, 1)    # weights sum to 1.00

priority = 'high' if severity >= 0.75 or (audio_score >= 0.6 and pir)
          else 'normal'

```

- This is a linear weighted-sum (late) fusion. The weights are declared in the docstring as prototype values to be tuned against Phase-1 bench data - do not claim they are learned or optimal.
- Why fuse at all: 'a scream' and 'a scream at 1 a.m. in an unlit lane with someone moving nearby' are different events deserving different urgency. Context is free (the sensors cost ~50 rupees) and it converts a binary detector into a graded one.
- Alternatives: a small trained classifier or logistic regression over the same features (needs labelled incident data nobody has yet); Dempster-Shafer or Bayesian belief fusion (principled about conflicting evidence, much harder to explain and tune); fuzzy inference rules. A transparent weighted sum was chosen because every dispatch decision must be explainable after the fact - and it is: Severity.reasons carries the full trace into the log and the dashboard.

5.8 The two-threshold dispatch gate

THE DECISION

```
dispatch if audio_score >= VERIFY_THRESHOLD (0.50)
          and severity   >= DISPATCH_THRESHOLD (0.60)
                                     hub/pipeline.py:104, hub/config.py:64

If no clip arrived within CLIP_WAIT_S (8 s):
    audio_score = stage1_confidence * 0.6          hub/pipeline.py:93
```

- Two gates, not one, and deliberately on different quantities: the first says 'the sound really was distress', the second says 'the whole situation warrants an aircraft'. A loud verified scream at noon on a busy road can pass gate 1 and still fail gate 2.
- The 0.6 haircut when no clip arrives is a fail-degraded design, not fail-open: the system keeps working with LoRa alone, but needs a much stronger Stage-1 confidence to fire. A Stage-1 confidence of 0.83 is needed just to reach the 0.50 verify threshold.
- Tuning direction: lower thresholds = higher recall, more false dispatches; higher = the opposite. Because a false dispatch has a real cost (battery, airspace, public trust), the current values lean conservative. This is where a ROC curve from Phase-1 data will set the final numbers.

5.9 Packet security - AES-128-CTR + HMAC-SHA256 + replay counter

The threat is concrete: a spoofed 25-byte radio packet would launch an aircraft. So the packet is authenticated first and decrypted second.

PACKET CRYPTO

Wire format - 25 bytes, fits one LoRa frame comfortably (hub/packets.py header):

offset	size	field	
0	2	magic 'VK'	} cleartext header
2	1	version = 1	} 9 bytes
3	2	node_id (uint16 big-endian)	} selects the key
5	4	counter (uint32 big-endian)	} CTR nonce + replay protection
9	8	AES-128-CTR ciphertext of:	
		event uint8 (1=scream 2=help 3=cry 4=crash)	
		conf uint8 (confidence * 255)	
		pir uint8 (0/1)	
		light uint8 (0..255, 0 = dark)	
		batt uint8 (%) + 3 reserved bytes	
17	8	MAC = HMAC-SHA256(node_key, header ciphertext)[:8]	

Per-node key derivation (so provisioning a node needs only the master key + its id):

```
node_key = HMAC-SHA256(master_key, 'node:<id>')[:16]
```

CTR initial value (a unique-per-packet nonce built from the cleartext header):

```
iv = 'VK' || version || node_id || counter || 7 zero bytes (16 bytes)
```

Verification order in unseal() - hub/packets.py:90:

1. length == 25 ?
2. magic and version ?
3. hmac.compare_digest(mac, expected) <- CONSTANT TIME, no timing leak
4. counter > last_counter ? <- replay rejected
5. only now decrypt

The design questions a security-minded examiner will ask

- Why CTR and not CBC or ECB? ECB leaks structure (identical plaintext gives identical ciphertext) and is never acceptable. CBC needs padding, which would push an 8-byte payload to 16 bytes - a 47% larger radio packet. CTR is a stream mode: the ciphertext is exactly as long as the plaintext, and it needs no padding.
- Why is the counter in cleartext? Because it is the CTR nonce and the replay key - the receiver needs both before it can do anything. It reveals only how many alerts a node has ever sent, which is not sensitive.

- Why encrypt-then-MAC and verify the MAC first? This is the provably correct composition order. Verifying before decrypting means a forged packet never reaches the AES code at all.
- Why truncate the HMAC to 8 bytes / 64 bits? Radio airtime. A forger gets one chance in 2^{64} per attempt; at LoRa SF9 data rates, brute force is not a practical attack. The trade is explicit, not accidental.
- Why is CTR nonce reuse the one thing that must never happen? Reusing (key, nonce) in a stream cipher lets an attacker XOR two ciphertexts and cancel the keystream. That is exactly why the counter is stored in NVS (node.ino:124) and must be monotonic - a counter that reset to 0 on reboot would both break replay protection and reuse nonces.
- Alternatives: AES-GCM or AES-CCM (single-pass authenticated encryption, standard, and what a v2 should use - CCM is what LoRaWAN itself uses); ChaCha20-Poly1305 (faster in software, no AES hardware needed); plain LoRaWAN network security (would require a LoRaWAN stack and a network server). Hand-composed CTR+HMAC was chosen because mbedtls on the ESP32 and PyCryptodome on the Pi both expose exactly these primitives, making the 40 lines on each side auditable and provably identical - and tests/test_hub.py proves round-trip, tamper-rejection, wrong-key-rejection and replay-rejection.
- The honest weakness: the default master key is a development key (000102...0e0f) hard-coded in both node.ino and hub/config.py. In deployment it must come from HUB_MASTER_KEY and be provisioned per installation. Say this before they say it.

5.10 Geodesy - haversine distance

Where flight_core/mavlink_interface.py:49 (used by the geofence, ETA, waypoint arrival and stall detector) and hub/sim_drone.py:18 (nearest-station choice).

HAVERSINE

```
R = 6,371,000 m    (mean Earth radius)
a = sin^2(dlat/2) + cos(lat1) * cos(lat2) * sin^2(dlon/2)
d = 2 * R * arcsin( sqrt(a) )
```

- Why haversine and not Euclidean on raw degrees: one degree of latitude is ~111 km but one degree of longitude shrinks with $\cos(\text{latitude})$. Treating degrees as a flat plane would put the geofence radius wrong by ~7% at Nagpur's latitude (21 N).
- Why not Vincenty or geographiclib (ellipsoidal, millimetre-accurate)? Haversine's spherical-Earth error is ~0.3% - about 15 m over 5 km. The waypoint tolerance is 5 m and the geofence is 5 km, so ellipsoidal precision would be invisible while adding a dependency. Note the numerically-stable arcsin form is used, not the arccos form which loses precision for small distances.

5.11 Nearest-station dispatch (the fleet selection rule)

Where hub/sim_drone.py:174 - DroneFleet._nearest(). This is the single place where distance ranking happens, by design, so the rule can be swapped in one spot.

DISPATCH SELECTION

```
chosen_station = argmin over stations of haversine(station.base, incident)
```

Four stations covering Nagpur (hub/config.py:DEFAULT_DRONE_BASES):

GHRCE (West)	21.1051, 79.0036	<- project base
Sadar (North)	21.1720, 79.0900	
Pardi (East)	21.1500, 79.1300	
Manish Nagar (South)	21.0930, 79.0680	

Ranking is by FIXED base position, not live position. Consequences, all intended:

- a given area always maps to the same drone (deterministic, demonstrable)
- a far idle drone is never sent when a near one exists
- the nearest drone wins even if busy: it cancel-replaces and relauches from wherever it currently is, so no drone is 'used up' after one flight

- Why straight-line distance rather than a road-network shortest path (Dijkstra/A*)? A quadcopter flies direct - it is not on the road graph. Straight-line IS the true path length here. That is the correct answer if someone asks

'where is your shortest-path algorithm?'

- Where a real graph search WOULD belong: routing around no-fly zones, which is exactly what `flight_core/obstacle_avoidance.py` does geometrically (next section). If keep-out zones ever became dense enough, a visibility graph plus A*, or RRT*, would replace it.
- Alternative selection rules explicitly considered: ranking by live position (rejected - non-deterministic, and area-to-drone mapping would shift between demos), skipping busy drones (rejected - would send a far drone while a near one is finishing), and weighting by battery state (a reasonable future refinement, and the docstring names it as the place to add it).

5.12 Map-based obstacle avoidance (pure geometry, no sensors)

Where `flight_core/obstacle_avoidance.py`. 7 dedicated unit tests, no SITL needed.

OBSTACLE AVOIDANCE

Step 1 - project lat/lon to a local tangent plane (equirectangular, exact enough over a few km):

```
east = radians(lon - lon_ref) * R * cos(radians(lat_ref))
north = radians(lat - lat_ref) * R
```

Step 2 - distance from an obstacle centre `p` to the flight leg `a->b`:

```
t = ((p - a) . (b - a)) / |b - a|^2      # projection parameter
tc = clamp(t, 0, 1)                       # clamp to the segment
dist = |p - (a + tc*(b - a))|
blocked if dist < obstacle.radius + clearance    (clearance default 8 m)
```

Step 3 - if blocked, insert two detour waypoints that bulge around the zone:

```
u      = unit vector along the leg
n      = unit vector from the obstacle centre toward the leg (bulge side)
offset = radius + clearance + max(3, clearance/2)    # lateral standoff
along  = radius + clearance                          # fore/aft standoff
wp1 = proj - u*along + n*offset , wp2 = proj + u*along + n*offset
```

Step 4 - recurse on (start -> wp1) and (wp2 -> end) so other zones are also cleared. Recursion depth is capped at 6 to guarantee termination.

- Why bulge toward the side the path already favours (`n` points from the centre toward the leg)? It gives the shorter of the two possible detours, and it is deterministic - the same input always produces the same route, which is what makes it unit-testable.
- With no obstacles configured, `plan_route()` returns `[end]` - byte-for-byte the same behaviour as a direct goto. That is deliberate: the nominal mission is unchanged when no map is loaded.
- Be precise about the limitation, because this is a favourite trap: this is MAP-BASED avoidance of operator-configured cylindrical keep-out zones. It is NOT sensor-based reactive avoidance - nothing in the repo reads a rangefinder or depth camera. Real reactive avoidance needs a lidar/rangefinder plus ArduPilot 4.x's own object-avoidance (BendyRuler / Dijkstra inside the autopilot), and it is on the roadmap. The module docstring states this in its first paragraph.

5.13 Verified mode transition - the core safety contribution

Where `flight_core/mission_executor.py:504 _set_mode_confirmed()`, `:541 _raw_set_mode()`

The bug that created this: ArduCopter 3.3 in SITL SILENTLY ignores dronekit's high-level mode setter under some conditions. The client believes it is in GUIDED; the autopilot is still in STABILIZE. A fire-and-forget command architecture would then 'take off' an aircraft that never left the ground - or fail to abort one that is airborne.

VERIFIED COMMAND DELIVERY

```
def _set_mode_confirmed(mode, timeout):
    vehicle.mode = VehicleMode(mode)          # 1. dronekit setter
    while time < deadline:
        if vehicle.mode.name == mode:          # 2. CONFIRM from HEARTBEAT
            return True
        if 700 ms elapsed since last raw send:
            COMMAND_LONG(MAV_CMD_DO_SET_MODE, base_mode, mode_id) # 3. raw
            SET_MODE(base_mode, mode_id)         # 4. backup
            vehicle.mode = VehicleMode(mode)     # re-poke
        sleep 0.2 s
    return False                               # caller must handle failure
```

```
Abort-path cross-fallback (mission_executor.py:726 _abort):
    if RTL will not confirm -> try LAND
    if LAND will not confirm -> try RTL
    then BLOCK until disarmed (up to 240 s) before returning to the queue
```

SAY IT LIKE THIS

"Every flight-mode change is treated as a request, not a fact. It is re-issued as raw MAVLink every 700 milliseconds until the autopilot's own heartbeat reports the new mode back to us. Nothing in the mission proceeds on an unconfirmed command. The rule in the codebase is: never use `vehicle.mode = ...` directly, anywhere - including on the emergency paths, which is where the original code was weakest."

- This is the subject of patent draft 1 (docs/patents/PATENT_1_VERIFIED_DISPATCH.md). The prior-art scan (US10216181B2, US10089889B2, US12184803B2) found that raw 'trigger -> fly to GPS' is NOT novel; the verification and interlock layer is the claimable part.
- Directly unit-tested without any simulator, using a `ModeRejectingVehicle` mock: `tests/test_units.py` `test_retries_raw_mavlink_until_telemetry_confirms` and `test_returns_false_when_mode_never_adopted`.

5.14 Failsafe arbitration lattice

Where `flight_core/failsafe_handler.py` - a 1 Hz daemon thread; `_emit()` at line 72 is the arbiter.

Failsafe	Condition	Action	Why that action
link_loss	MAVLink heartbeat age > 10 s (LINK_LOSS_TIMEOUT)	RTL	If the heartbeat is stale, every other reading is frozen data - battery reads 'fine', GPS reads 'locked'. None of it can be trusted, so come home.
critical_battery	level <= 10% (CRIT_BATTERY_PCT)	LAND	There may not be enough energy to reach home. Down now, wherever we are, beats falling out of the sky en route.
low_battery	level <= 20% (LOW_BATTERY_PCT)	RTL	Enough margin to return.
gps_loss	fix_type < 2 for 3 CONSECUTIVE 1 Hz samples	LAND	With no position fix, RTL literally cannot navigate. LAND in place is the only safe autonomous response.
geofence_breach	haversine(home, position) > 5000 m	RTL	Regulatory and operational boundary.
mission_timeout	elapsed > 1800 s (MAX_MISSION_DURATION)	RTL	Bounded flight time; catches any stuck state the phase logic missed.

ARBITRATION

Arbitration rules (failsafe_handler.py:72 _emit):

1. LAND outranks RTL, and required_action is NEVER downgraded once LAND
2. each named failsafe fires at most once per mission ...
3. ... except that an RTL-severity name may re-fire to ESCALATE to LAND
4. GPS loss is DEBOUNCED: 3 consecutive bad samples, streak reset on recovery
5. the RTL phase loop re-checks the arbiter every second and switches to LAND mid-return if the demand escalates (mission_executor.py:_rtl_and_wait_landed)
6. the monitor thread wraps every iteration in try/except - it must never die

- Why debounce GPS specifically? A single bad sample is a glitch; putting an aircraft down on one noisy reading would be the failsafe causing the accident. Three consecutive seconds is evidence.
- Why 'fires once per name'? Without it, a 19% battery emits an event every second for the rest of the flight, burying the log and the operator's attention.
- Why does absent battery telemetry warn loudly instead of being silent? Because on real hardware it means an unconfigured power module - i.e. the battery failsafe is INACTIVE. Silence there would be dangerous. failsafe_handler.py:_check_battery, tested by test_absent_battery_telemetry_warns_but_does_not_trigger.
- This is patent draft 2 (docs/patents/PATENT_2_FAILSAFE_ARBITER.md).

5.15 Smaller algorithms worth knowing

Algorithm	Formula / rule	Where	Why
Leg stall detector	Track best_dist. Progress = d < best_dist - 2 m. If no progress for 45 s -> raise, mission fails to RTL.	mission_executor.py: 596-616	Wind, a mode flip or a dropped command can leave the aircraft hovering forever. Fails in 45 s instead of burning battery until the 1800 s global timeout.
Takeoff completion	alt >= 0.95 * target_alt	mission_executor.py: _arm_and_takeoff	Copter never settles exactly at the commanded altitude; a strict equality would hang forever.
Waypoint arrival	haversine(current, target) <= 5 m (WAYPOINT_TOLERANCE)	mission_executor.py: _goto_waypoint	GPS noise alone is 1-3 m. 5 m is achievable; the e2e test measured 0.6 m closest approach.
Kit drop altitude	descend to 3 m, alt - 3 <= 0.7 m, fire servo, climb back to cruise	mission_executor.py: _deliver_kit	Low enough that the kit is not damaged, high enough to stay clear of people and ground effect.
Priority ordering	key = (priority_rank, queued_at); rank = critical 0, high 1, normal 2, low 3	trigger_api/mission_queue.py:27, 153	Stable ordering: higher priority first, FIFO within a priority. Sorting on insert keeps the pop O(1).
ETA estimate	eta = haversine(current_or_home, target) / cruise_speed + 20 s	trigger_api/main.py:_eta_seconds	The +20 s covers arm and climb. Straight-line at cruise speed is the honest estimate for a quadcopter.
Telemetry bandwidth	Send the full breadcrumb path only every 4th WebSocket frame; drop it otherwise.	trigger_api/main.py:w s_telemetry	The path is up to 500 points and grows by one per tick. Resending it at 2 Hz would dominate a real telemetry link.
Mission generation counter	Each dispatch bumps _gen; a leg loop that sees a changed _gen exits silently.	hub/sim_drone.py:35, 111	Lets a new alert cancel-and-replace a running simulated mission with no locks held across sleeps and no orphaned threads.
Atomic registry write	Write nodes.json.tmp, then os.replace()	hub/node_registry.py: save	os.replace is atomic on both POSIX and Windows, so a crash mid-write cannot corrupt the replay-counter state.
Crash recovery	On boot, any mission still 'queued'/'running' in SQLite is marked 'interrupted'.	trigger_api/store.py:_mark_interrupted	Those rows belong to a process that no longer exists; leaving them would misreport a flight as in progress.

6 Master comparison table - every choice, its alternatives, and why

This is the table to memorise. If you can give the 'Why this one' column for any row, you can defend the whole design. Rows are grouped by layer.

Layer	Decision point	What we use	Where in the code	Alternatives considered	Why this one
SENSIN G	Microphone	INMP441 I2S MEMS digital mic	firmware/node/node.ino:43	Analog electret + MAX9814 amp; MAX4466; USB mic	The audio path is digital from the mic die onward, so a pole-mounted node with long power runs cannot inject analog hum into the signal. 24-bit, needs no analog front end, ~200 rupees.
SENSIN G	Node MCU	ESP32-S3 (240 MHz dual core, PSRAM, vector extensions)	whole node sketch	ESP32 classic; Arduino Nano 33 BLE Sense; STM32 + mic; Raspberry Pi Zero at every node	The S3 adds the SIMD/vector instructions TFLM uses and can address PSRAM for the 128 KB audio ring buffer. Classic ESP32 lacks both. A Pi per pole would triple cost and power for no detection benefit.
SENSIN G	Feature front end	Hand-written MFCC, identical in Python and C	ml/mfcc.py + stage1.cpp:90	librosa MFCC; log-mel spectrogram; Edge Impulse generated DSP; learned front end	Training/deployment feature parity is the single most common cause of a model that works in the lab and fails on the device. One hand-written definition, mirrored step for step, plus tests/test_mfcc.py, makes parity checkable.
SENSIN G	Stage-1 model (deployed)	MFCC mean+std (26) -> Dense 24 ReLU -> 4 softmax; 748 params, ~3 KB	firmware/node/stage1_nn.h	int8 CNN under TFLM (code path exists); SVM; GMM; decision tree; keyword-spotting DS-CNN	It trains with NumPy alone, deploys as three float matmuls with NO ML library on the device, and is small enough to be obviously real-time. The CNN is strictly better on accuracy and is the next step - it is written, not yet trained.
SENSIN G	Where Stage 1 runs	On the node, on-device	node.ino:210	Stream all audio to the hub and classify centrally	Streaming continuous audio needs WiFi/4G at every pole (cost, power, coverage) and turns every node into a live microphone - a privacy problem. On-device means only ~0.2% of audio is ever transmitted.
SENSIN G	Alert radio	LoRa SX1278, 433 MHz, SF9	node.ino:189, gateway.ino:26	GSM/4G module; NB-IoT; WiFi mesh; Zigbee; LoRaWAN with a network server	5-10 km per hop with no SIM, no subscription and no internet dependency - it keeps working in exactly the low-infrastructure areas the project targets. SF9 balances range against airtime.
SENSIN G	Packet size	25 bytes fixed	hub/packets.py:38	JSON over LoRa; include GPS coordinates in the packet	LoRa airtime is the scarce resource. The node sends only its id; the hub's registry supplies the surveyed coordinates - so no node needs a GPS module (saves cost, power and a fix-acquisition delay).
SENSIN G	Node crypto	AES-128-CTR + truncated HMAC-SHA256 + monotonic counter	hub/packets.py, node.ino:120	AES-GCM/CCM; ChaCha20-Poly1305; LoRaWAN link security; no encryption	A spoofed packet launches an aircraft, so authentication is mandatory. mbedtls (ESP32) and PyCryptodome (Pi) both expose exactly these primitives, so the two 40-line implementations are auditable and provably identical. GCM/CCM would be the cleaner v2.
SENSIN G	Replay defence	Per-node uint32 counter persisted in NVS; reject counter <= last seen	node.ino:124, packets.py:103	Timestamps (needs synchronised clocks); challenge-response (needs a downlink)	LoRa here is one-way and the nodes have no RTC, so a monotonic counter is the only mechanism that needs neither. Persisting it in NVS is what makes it survive power loss on a solar node.
HUB	Hub computer	Raspberry Pi 5	hub/ package	Pi 4; Jetson Nano; a cloud server; an industrial PC	Enough CPU to run PANNs on 4-second clips locally, so no audio ever leaves the site and the system survives an internet outage. Jetson would add GPU cost the workload does not need.
HUB	Stage-2 model	PANNs CNN14, pretrained on AudioSet (527 classes)	hub/verifier.py:80	YAMNet; AST transformer; train our own from scratch; fine-tuned PANNs	AudioSet is 2 million labelled clips - orders of magnitude more supervision than this project could ever collect, and it already contains Screaming/Shout/Yell/Crying classes. Fine-tuning it on local data is the correct next step.

Layer	Decision point	What we use	Where in the code	Alternatives considered	Why this one
HUB	Stage-2 fallback	Energy heuristic: $0.45 \cdot \text{loudness} + 0.35 \cdot \text{highness} + 0.20 \cdot \text{burstiness}$	hub/verifier.py:60	Refuse to run without torch; ship a small trained model	It lets the entire chain be demonstrated and unit-tested on any laptop without a 2 GB install. It is labelled 'dev fallback' in code, in logs and in the docs, and is never presented as an accuracy result.
HUB	Evidence fusion	Linear weighted sum: $0.60 \text{ audio} + 0.15 \text{ stage1} + 0.10 \text{ PIR} + 0.08 \text{ dark} + 0.07 \text{ night}$	hub/fusion.py:35	Trained classifier; logistic regression; Dempster-Shafer; Bayesian network; fuzzy rules	Every dispatch must be explainable after the fact, and Severity.reasons carries the full trace. Also: nobody has labelled incident data yet, so a learned fuser would have nothing to learn from. Weights are declared as prototype values.
HUB	Decision rule	Two independent thresholds (audio 0.50 AND severity 0.60)	hub/pipeline.py:104	Single threshold; ML-learned decision boundary; human dispatcher in the loop	The two gates test different things - 'was it really distress' and 'does the situation warrant an aircraft'. A human in the loop is a legitimate alternative and would raise precision, at the cost of the response latency the project exists to cut.
HUB	Missing-clip behaviour	Degrade: $\text{audio_score} = \text{stage1_confidence} \cdot 0.6$	hub/pipeline.py:93	Fail closed (never dispatch); fail open (trust Stage 1)	Fail-degraded: the system keeps working on LoRa alone, but a Stage-1 confidence of 0.83+ is now needed to clear the gate. Neither blind nor blind-trusting.
API	Web framework	FastAPI + uvicorn	trigger_api/main.py	Flask; Django REST; raw HTTP server; gRPC	Pydantic validation at the edge is the point: bounds are enforced in models.py so the flight core can never receive an impossible altitude or coordinate. Native async also gives the WebSocket telemetry stream for free.
API	Input validation	Pydantic v2 field bounds + a geofence check at the endpoint	models.py, main.py: <code>_ensure_inside_geofence</code>	Validate inside the flight core; validate in the UI only	Reject an out-of-fence target with HTTP 400 on the ground, rather than letting the geofence failsafe abort it mid-air. Fail at the edge, not in flight.
API	Queue	Single-drone serial priority deque, depth cap 20, HTTP 429 beyond	mission_queue.py	Celery/Redis; RabbitMQ; run missions concurrently	There is one physical aircraft, so concurrency is physically impossible - a broker would add operational weight for nothing. The cap makes overload an explicit, logged 429 instead of unbounded memory growth.
API	Persistence	SQLite, best-effort, failures logged and swallowed	trigger_api/store.py	PostgreSQL; a JSON file; no persistence	Zero-configuration, single-file, already in the standard library - right for an embedded companion computer. Critically, a storage failure must never take down the dispatch path, so every write is wrapped.
FLIGHT	Autopilot	ArduPilot Copter (≥ 4.3 real, copter-3.3 in SITL)	sitl/, run_all.ps1	PX4; Betaflight; a custom controller	ArduPilot has the most mature autonomous-mode and failsafe stack and a first-class MAVLink surface. PX4 would work with minor mode-name changes. Writing our own control loops would be reckless.
FLIGHT	MAVLink library	dronekit 2.9.2 (with a Python 3.10+ compatibility shim)	flight_core/mavlink_interface.py	pymavlink directly; MAVSDK-Python; ROS2 + MAVROS	dronekit's Vehicle abstraction made the state machine readable fast. It is UNMAINTAINED (last release targeted Python 2/3.7) and needs a collections-ABC shim plus the 'future' package - migrating to pymavlink is the number-one roadmap item. Say this proactively.
FLIGHT	Command delivery	Verified mode transition: re-send raw MAVLink every 700 ms until HEARTBEAT confirms	mission_executor.py:504	Fire and forget (what dronekit does by default); ack-based only	ArduCopter 3.3 SITL silently ignores dronekit's plain mode setter - the client believes GUIDED while the autopilot stays STABILIZE. This is the project's core safety contribution and patent draft 1.
FLIGHT	Abort semantics	Landing interlock - block until landed AND disarmed (≤ 240 s) before the queue resumes	mission_executor.py:726	Return immediately after commanding RTL	Returning early would let the queue start the next mission against an airborne vehicle. The invariant is absolute: the queue can never fly against an armed aircraft (also enforced pre-flight at <code>_wait_until_safe_to_start</code>).
FLIGHT	Failsafe policy	Arbitration lattice: LAND > RTL, never downgraded, debounced GPS, one event per name	failsafe_handler.py:72	First-event-wins; last-event-wins; simple flag per condition	Prevents two catastrophic classes of bug: a low-battery RTL overwriting a critical-battery LAND, and a single noisy GPS sample putting the aircraft down. Patent draft 2.

Layer	Decision point	What we use	Where in the code	Alternatives considered	Why this one
FLIGHT	Obstacle avoidance	Deterministic geometric routing around configured cylindrical keep-out zones	obstacle_avoidance.py	Sensor-based reactive OA (rangefinder/depth camera + ArduPilot BendyRuler/Dijkstra); RRT*; potential fields	Pure geometry needs no extra hardware, is fully unit-testable without SITL, and reduces exactly to a direct goto when no map is loaded. Sensor-based OA is the honest roadmap item - the module says so in its docstring.
FLIGHT	Payload release	SG90 servo via MAV_CMD_DO_SET_SERVO on AUX 1 (channel 9), 1900 open / 1100 hold	payload_release.py, config.py	Electromagnet; solenoid; parachute drop; landing to hand over	A raw MAVLink command works identically in SITL and on hardware with no dronekit-specific API. Design rule: a failed release NEVER causes a loiter - the aircraft proceeds to RTL and reports the failure.
FLIGHT	Evidence camera	picamera2 + H264Encoder at 4 Mbit/s during the hover window; no-op if absent	camera_recorder.py	USB webcam + OpenCV; GoPro; live RTSP/WebRTC stream to authorities	The native Pi camera stack is hardware-encoded, so the CPU stays free for the flight loop. Every call is a logged no-op when picamera2 is missing - camera trouble can never block or fail a mission. Live streaming is the named future work.
UI	Dashboard	React 18 + Vite + Leaflet with offline tile assets	dashboard/src/	Mission Planner / QGroundControl; Grafana; plain HTML	The audience is a police/security operator, not a UAV pilot: one map, one incident list, dispatch and recall. Deliberately exposes NO flight controls beyond dispatch/cancel. Leaflet is dependency-light and works offline.
TEST	Verification strategy	80 fast unit tests with mocked vehicles + one real end-to-end SITL flight	tests/	Only manual flight testing; only unit tests; hardware-in-the-loop rig	Safety logic (mode confirmation, abort interlock, arbitration) is tested against a ModeRejectingVehicle mock in milliseconds - a simulator cannot reproduce a rejecting autopilot on demand. The 5-6 minute SITL flight then proves the whole chain.
DEMO	No-hardware demo path	Phone browser -> /phone-alert -> real Stage 1 + Stage 2 -> animated fleet on the map	hub/webapp.py, sim_drone.py	Wait for hardware; pre-recorded video; SITL only	It exercises the REAL detection and decision code on REAL audio from a phone mic, so the pipeline is proven before any hardware exists. It explicitly does not test LoRa range, the ESP32, or physical flight - and the docstring says exactly that.

7 Every library: version, role, and the alternative

Library	Version	Side	Used in	What it does for us	Alternative / note
dronekit	2.9.2 (pinned)	Python	flight_core/mavlink_interface.py, mission_executor.py	High-level Vehicle object: mode, armed, location, battery, gps_0, simple_takeoff, simple_goto, parameters	UNMAINTAINED - last released for Python 2/3.7. Needs the collections-ABC shim plus the 'future' package. Migrating to pymavlink directly is roadmap item #1.
dronekit-sitl	3.3.0	Python	tests, run_all.ps1, sitl/	Spawns a simulated ArduCopter so the whole stack can fly with no aircraft	Only the 'copter-3.3' build works on Windows - plain 'copter' fails. Alternative: ArduPilot's own sim_vehicle.py (4.x), which is the upgrade path.
pymavlink	>= 2.4.40	Python	mission_executor._raw_set_mode, payload_release.set_servo	Raw MAVLink message construction: COMMAND_LONG, SET_MODE, DO_SET_SERVO - the fallback path dronekit cannot express	This is what a full migration would use everywhere.
future	>= 0.18.3	Python	transitively, for dronekit	Python-2 compatibility shims dronekit still imports	Removed once dronekit is.
FastAPI	>= 0.110	Python	trigger_api/main.py, hub/webapp.py	HTTP routes, dependency injection for the X-API-Key guard, WebSocket endpoint, automatic OpenAPI docs at /docs	Flask (no async, no built-in validation), Django REST (far heavier).
pydantic	>= 2.6	Python	trigger_api/models.py	Declarative bounds checking - lat/lon ranges, altitude 2-120 m, hover 0-3600 s, priority enum - enforced before the flight core sees anything	Hand-written validation; marshmallow. v2 syntax is used (field_validator).
uvicorn[standard]	>= 0.27	Python	run scripts, hub/main.py	ASGI server hosting both FastAPI apps	hypercorn; gunicorn + uvicorn workers.
websockets	>= 12.0	Python	WS /ws/telemetry	Pushes a telemetry snapshot every 500 ms to the dashboard	Server-sent events; HTTP polling (what the demo dashboard actually falls back to).
requests	>= 2.31	Python	hub/dispatcher.py, scripts/demo_phase0.py	The one HTTP call that matters: POST /trigger with a 10 s timeout	httplib (async); urllib.
numpy	>= 1.26	Python	ml/*, hub/verifier.py, hub/webapp.py	All array maths: FFT, mel filterbank, DCT, the entire MLP forward and backward pass, WAV decoding	Not optional anywhere. It is the only hard dependency of the committed Stage-1 model.
pycryptodome	>= 3.20	Python	hub/packets.py	AES-128 in CTR mode (Crypto.Cipher.AES)	cryptography (heavier, also fine); the stdlib has no AES. HMAC-SHA256 comes from the stdlib hmac + hashlib.
pyserial	>= 3.5	Python	hub/lora_gateway.py	Reads the gateway ESP32's USB serial stream at 115200	Only needed on the real hub; SimGateway replaces it in tests.
panns-inference + torch	optional, Pi only	Python	hub/verifier.py:PannsBackend	Stage-2 AudioSet tagging with CNN14 - 527 clipwise class probabilities	~2 GB install, so it is deliberately optional: the import is inside __init__ and any failure falls back to the heuristic. Alternatives: YAMNet (TF), AST.
picamera2	optional, Pi only	Python	flight_core/camera_recorder.py	Hardware H.264 recording of the hover window to mp4	OpenCV + USB camera. Absent = logged no-op, never a mission failure.
tensorflow	optional	Python	ml/train_stage1.py, ml/train_gpu.py	Trains the CNN and does int8 post-training quantisation to .tflite	PyTorch + ONNX + a converter. Optional on purpose - train_stage1_numpy.py needs none of it.
librosa / soundfile	optional	Python	ml/train_stage1.py, ml/train_gpu.py	Robust audio loading and resampling of mp3/ogg/flac datasets	Used ONLY for file loading, never for features - features always come from ml/mfcc.py.

Library	Version	Side	Used in	What it does for us	Alternative / note
scikit-learn	optional	Python	ml/train_gpu.py	classification_report and confusion_matrix for honest metrics	Hand-computed metrics.
pytest	>= 8.0	Python	tests/	The 80-test unit tier; pytest.ini pins testpaths	unittest.
fpdf2	2.8.7	Python	docs/build_costing_pdf.py, docs/build_seminar_guide_pdf.py	Generates the PDF deliverables directly, with full table-layout control	reportlab; LaTeX; markdown -> pandoc.
matplotlib	>= 3.7	Python	docs/build_diagrams.py, build_v2_figures.py	Generates all the original figures for the paper and thesis	draw.io; TikZ.
python-docx	>= 1.0	Python	docs/build_docx.py	Renders the markdown sources into Word papers/thesis	pandoc.
LoRa (Sandeep Mistry)	Arduino lib	C++	firmware/node/node.ino, firmware/gateway/gateway.ino	SX1278 driver: begin(freq), setSpreadingFactor, beginPacket/write/endPacket, parsePacket, packetRssi	RadioLib (more capable, more complex); Semtech's own driver; a LoRaWAN stack (LMIC).
mbedtls	bundled in ESP-IDF	C	firmware/node/node.ino:110-140	mbedtls_aes_crypt_ctr (AES-128-CTR) and mbedtls_md_hmac (HMAC-SHA256) - the exact mirror of the Python side	tinycrypt; a hand-rolled AES (never do this).
driver/i2s.h	ESP-IDF	C	firmware/node/node.ino:64	I2S peripheral in master-RX mode: 16 kHz, 32-bit slots, left channel only, 4 DMA buffers of 512 samples	Arduino's I2S wrapper class; bit-banging (not viable at 16 kHz).
Preferences (NVS)	ESP-IDF/Arduino	C++	firmware/node/node.ino:124, 180	Persists the alert counter across reboots and power loss - without it, replay protection would break on every restart	EEPROM emulation; an external FRAM.
WiFi + HTTPClient	Arduino core	C++	firmware/node/node.ino:155	POSTs the 4 s WAV clip to the hub's /clip/{id}/{ctr} endpoint	ESP-NOW (lower power, needs a bridge); MQTT.
TensorFlow Lite Micro	optional, Arduino lib	C++	firmware/node/stage1.cpp:114 (USE_TFLM_STAGE1)	Runs the int8 CNN on-device with a 40 KB tensor arena and a MicroMutableOpResolver of 8 ops	Chirale_TensorFlowLite or esp-tflite-micro. Avoided entirely in the deployed MLP build - that is the point of the MLP.
React	18.3	JS	dashboard/src/	Component state for the live telemetry view	Svelte; Vue; vanilla JS (which is what the hub's own server-rendered pages use).
Vite	5.4	JS	dashboard/	Dev server on port 5173 and the production bundle	Create React App (deprecated); webpack.
Leaflet + react-leaflet	1.9 / 4.2	JS	dashboard/src/Map.jsx	The live map: node markers, drone position, breadcrumb path, follow toggle	Mapbox GL / Google Maps (both need an API key and internet - Leaflet works with offline tiles).

8 Numbers cheat sheet - every constant they can ask about

If a panel member points at a number in your slides, it is in this table with the file that owns it and the reason it has that value.

Audio and model

Constant	Value	Where	Why that value
Sample rate	16,000 Hz	ml/mfcc.py:21, node.ino:46	Nyquist gives 8 kHz of bandwidth - human screams peak around 1-4 kHz, so this covers the whole informative band at a quarter of CD data rate.
Inference window	2.0 s (32,000 samples)	ml/mfcc.py:22, node.ino:96	Long enough to contain a whole scream or the word 'bachao'; short enough that inference every 0.5 s is affordable.
FFT size	512 (32 ms frame)	ml/mfcc.py:24	Standard speech framing: short enough that the spectrum is quasi-stationary, long enough for ~31 Hz frequency resolution. A power of 2, for radix-2 FFT.
Hop	256 (50% overlap)	ml/mfcc.py:25	Overlapping frames prevent an event falling in a window boundary and being smeared by the Hamming taper.
Frames per window	$123 = 1 + (32000 - 512) / 256$	ml/mfcc.py:29	Derived, not chosen. Both the Python and C code compute it with the same expression.
Mel filters	40, spanning 0-8000 Hz	ml/mfcc.py:26	The usual choice for speech; enough resolution without over-fragmenting 257 FFT bins.
MFCC kept	13	ml/mfcc.py:27	Classic speech-recognition convention: the low DCT coefficients hold the spectral envelope, the high ones mostly noise.
Pre-emphasis	0.97	ml/mfcc.py:28	Standard first-order high-pass coefficient; compensates the natural -6 dB/octave rolloff of voiced speech.
Log epsilon	1e-6	ml/mfcc.py:29	Prevents $\log(0) = -\infty$ on a silent mel band. Identical in the C code (stage1.cpp:32).
Stage-1 features	26 (13 means + 13 stds)	ml/train_stage1_num.py.py:55	Pools 123 frames into a fixed-size vector so a tiny Dense layer suffices.
Hidden units	24	ml/train_stage1_num.py.py:44	Large enough for 4 classes on this feature set, small enough that the whole model is ~3 KB of floats.
Classes	4: background(0), scream(1), cry(2), help(3)	ml/train_stage1_num.py.py:41	Index 0 MUST be the negative class - the firmware tests 'cls != S1_BACKGROUND'.
Learning rate / epochs	3e-3 / 400, Adam	ml/train_stage1_num.py.py:105	Full-batch on a small dataset; converges well before 400 epochs.
Train/val split	85 / 15	ml/train_stage1_num.py.py:103	train_gpu.py additionally splits by FILE so augmented copies of one clip cannot straddle the split (data leakage - reviewers check for this).
Augmentation	gain 0.5-1.4x, time shift up to 0.5 s, noise mix at 0-20 dB SNR (75% of samples)	train_stage1_numpy.py:69, train_gpu.py:117	Teaches invariance to distance/volume, to where in the window the event falls, and to street noise - the three things that vary most in the field.
TFLM arena	40 KB	stage1.cpp:124	The scratch memory for intermediate tensors; tuned to the model and comfortably inside the S3's SRAM.
Exported model size	~33 KB int8	ml/train_gpu.py output	Measured in the smoke test recorded in the git history.

Node behaviour, radio, and hub decisions

Constant	Value	Where	Why
Node trigger score	0.60	node.ino:49	Tuned for RECALL, not precision - Stage 2 removes the false positives, so missing an event is the worse error here.
Inference cadence	every 16 frames (~0.5 s)	node.ino:97	Four inferences per 2 s window means an event is seen from several alignments; also bounds CPU and power.
Refractory period	15,000 ms	node.ino:214	One incident should produce one alert, not thirty. Prevents flooding both the LoRa channel and the drone queue.
Clip buffer	4 s in PSRAM (~128 KB)	node.ino:57	A circular buffer, so the uploaded clip contains audio from BEFORE the trigger - the onset is what Stage 2 needs most.
LoRa frequency	433 MHz	node.ino:38, gateway.ino:16	Licence-free ISM band in India; lower frequency = better building penetration and range than 868/915 MHz.
Spreading factor	9	node.ino:189, gateway.ino:26	Mid-point of the range/airtime trade. Node and gateway MUST match or nothing is received.
Packet length	25 bytes	hub/packets.py:38	9 header + 8 ciphertext + 8 MAC. Comfortably one frame at SF9.
MAC length	8 bytes (64 bits truncated)	hub/packets.py:105	Airtime economy; a forger has a $1\text{-in-}2^{64}$ chance per attempt.
Serial baud	115,200	hub/config.py:60, gateway.ino:21	Standard, fast enough for hex frames at any realistic alert rate.
VERIFY_THRESHOLD	0.50	hub/config.py:64	Minimum Stage-2 audio score to count as distress at all.
DISPATCH_THRESHOLD	0.60	hub/config.py:65	Minimum fused severity to commit an aircraft. Deliberately above the verify gate.
CLIP_WAIT_S	8.0 s	hub/config.py:66	Long enough for a WiFi upload of ~128 KB; short enough that a missing clip does not delay a real emergency.
Degraded score factor	0.6	hub/pipeline.py:93	Without Stage-2 evidence, Stage-1 confidence must reach 0.83 to clear the 0.50 gate.
High-priority rule	severity ≥ 0.75 , or audio ≥ 0.6 with PIR	hub/fusion.py:47	Corroborated motion is strong enough evidence to escalate on its own.
Night window	20:00 to 06:00	hub/fusion.py:33	The hours the project is most concerned with.
Cruise speed (fleet ETA)	15 m/s (~54 km/h)	hub/config.py:76	Typical delivery-quadcopter cruise; used for the REAL ETA shown on the dashboard even when the on-screen animation is compressed.

Flight parameters and failsafes

Constant	Value	Where	Why
Cruise altitude	15 m (env CRUISE_ALT)	flight_core/config.py:56	High enough to clear people, trees and small structures; low enough to keep the incident in camera frame.
Altitude bounds	2 m to 120 m	config.py:60, models.py:15	120 m is the small-UAS AGL ceiling in most jurisdictions, including DGCA rules; enforced at the API edge, not in flight.
Cruise speed	8 m/s	config.py:57	Conservative for the flight stack; docs recommend WPNAV_SPEED 5 m/s for first flights.
Hover duration	30 s default (0-3600 allowed)	config.py:58	The observation/evidence window. Also the camera recording window.
Waypoint tolerance	5 m	config.py:60	GPS noise is 1-3 m, so tighter would risk never 'arriving'. Measured 0.6 m closest approach in the e2e test.
Geofence radius	5,000 m	config.py:69	Rejected at the API edge with HTTP 400, and re-checked in flight by the arbiter.
Low / critical battery	20% / 10%	config.py:67-68	20% leaves margin to fly home; 10% means land now, wherever you are.
GPS debounce	3 consecutive bad 1 Hz samples	config.py:71	One glitch must never put an aircraft down; three seconds is evidence.

Constant	Value	Where	Why
Leg stall timeout	45 s with < 2 m progress	config.py:72	Detects a stuck leg long before the 1800 s mission timeout would.
Link-loss timeout	10 s heartbeat age	config.py:73	Stale telemetry means every other reading is frozen and untrustworthy.
Mission timeout	1,800 s (30 min)	config.py:70	Hard upper bound on any single flight.
Obstacle clearance	8 m lateral	config.py:74	Added to each keep-out zone's own radius before the leg is considered blocked.
Payload servo	channel 9 (AUX 1), 1900 open / 1100 hold	config.py:77-79	Standard hobby-servo PWM range; AUX OUT 1 maps to servo output 9 on a Pixhawk.
Drop altitude	3 m	config.py:80	Low enough not to damage the kit, high enough to stay clear of people and ground effect.
Mode confirm retry	raw MAVLink every 700 ms	mission_executor.py: 528	Faster than the ~1 Hz heartbeat, so several attempts land inside one confirmation window without flooding the link.
Abort land wait	240 s	mission_executor.py: ABORT_LAND_WAIT_S	Generous enough for an RTL from the geofence edge; bounded so the queue cannot deadlock.
Pre-flight disarm wait	120 s	mission_executor.py: PREFLIGHT_DISARM_WAIT_S	Then refuse to start rather than fly against an armed aircraft.
Queue depth / history	20 pending (429 beyond) / 1000 records	config.py:88-89	Bounded memory; overload becomes an explicit logged rejection.
Telemetry interval	500 ms	config.py:86	2 Hz is smooth on a map without saturating a real telemetry radio.
Takeoff threshold	95% of target altitude	mission_executor.py: _arm_and_takeoff	Copter never settles exactly at the commanded altitude.
Pre-arm wait	45 s, then proceed anyway	mission_executor.py: _arm_and_takeoff	is_armable mirrors EKF flags that are unreliable on Copter 3.3. The explicit arm confirmation is the real gate - do NOT 'fix' this by raising on the timeout.

9 Dataset, training protocol and what to claim about accuracy

THE MOST IMPORTANT SLIDE DISCIPLINE IN THIS PROJECT

Do NOT put an accuracy number on a slide that came from the bootstrap dataset. The committed model was trained on a synthesised bootstrap set whose only purpose is to prove the pipeline runs end to end. Saying '99% accuracy' from that data is the fastest way to lose a viva. Say instead: 'the pipeline is validated end to end; the detection metrics are the Phase-1 deliverable, and the trainer already reports precision, recall, F1, confusion matrix and background false-alarm rate.'

The four classes (docs/DATASET_AND_TRAINING.md)

Index	Class	What it is	Fires the drone?
0	background	traffic, crowd, wind, music, normal talking, silence	No - index 0 must be the negative class
1	scream	human screams and shouts of fear	Yes
2	cry	distress crying, sobbing, wailing	Yes
3	help	spoken calls: 'help', 'bachao', 'madad', 'save me'	Yes

Firmware event codes on the wire are 1=scream, 2=help_keyword, 3=cry, 4=crash. Note the class index and the event code are NOT the same numbering - hub/webapp.py:57 holds the EVENT_CODE mapping. 'crash' (4) has no training data yet and is reserved.

Where the audio comes from

- ESC-50 - 2000 labelled environmental clips, auto-downloaded by ml/train_gpu.py with no login. Supplies 'background' and, via its crying_baby category, 'cry'.
- A Kaggle human-scream dataset (needs a kaggle.json token) supplies 'scream'.
- Your own recordings in ml/data/<class>/*.wav, captured with ml/record_samples.py - this is what makes 'bachao' and 'madad' possible at all, since no public corpus contains them.
- ml/make_bootstrap_dataset.py synthesises a stand-in set so the pipeline can be trained and demonstrated before any of the above exists. This is what the committed stage1_nn.h came from.

The training protocol - and the two things reviewers check

1. Every clip is resampled to 16 kHz mono and reduced to one or two 2-second windows (centre, plus the tail if the clip is long enough) - ml/train_gpu.py:60.
2. Features come ONLY from ml/mfcc.py, never from librosa's MFCC - that is what guarantees the ESP32 sees the same features the model trained on.
3. Augmentation is applied to the TRAIN SET ONLY: random gain 0.6-1.4x, random time shift up to 0.5 s, and mixing in a real background clip at a random SNR between 0 and 20 dB for 75% of samples. The noise pool is drawn only from TRAIN backgrounds.
4. CHECK 1 - the split is by FILE, not by sample, so augmented copies of one recording can never appear on both sides of the split. This is the classic data-leakage mistake in audio ML, and ml/train_gpu.py:232 avoids it explicitly.
5. CHECK 2 - the reported metrics are per-class precision, recall and F1, a confusion matrix, and the background false-alarm rate - not a single accuracy figure. On a class-imbalanced problem, accuracy is a meaningless headline.
6. Export: int8 quantisation calibrated on ~200 real training samples, written as stage1_int8.tflite plus stage1_model_data.cc (a C byte array) for TFLM; or, on the NumPy path, stage1_nn.h with the raw float weights.
7. ml/eval_pipeline.py then evaluates the CASCADE, not just Stage 1: per-class recall, background false-trigger rate, Stage-2 score separation between distress and background, and what fraction of Stage-1 triggers the hub actually confirms. It runs the exported int8 model so the numbers describe the deployed weights, not the float

training model.

The metric to quote for a cascade

WHAT THE NUMBERS MEAN

End-to-end recall = $P(\text{dispatch} \mid \text{real incident})$
= $P(\text{stage1 fires}) * P(\text{stage2 confirms} \mid \text{stage1 fired})$
End-to-end precision = $P(\text{real incident} \mid \text{dispatch})$

Stage 1 is tuned for RECALL (threshold 0.60, high-recall/low-precision by design)

Stage 2 restores PRECISION.

So: a Stage-1 false positive costs one 4 s clip upload; a Stage-1 MISS costs the whole incident. That asymmetry is the entire justification for the cascade.

10 What is proven, what is not - answer this honestly

Verified working

Item	Evidence
80 unit tests pass, 0 failures	python -m pytest, run this session: '80 passed in 20.99s'. Note: two of them (tests/test_stage1_nn.py) self-skip if either the trained model or the generated dataset directory ml/data is missing, since both are gitignored artifacts - run 'python ml/make_bootstrap_dataset.py' once and the full 80 run.
Failsafe arbitration logic	13 dedicated tests: single-fire, LAND-never-downgraded, escalation, GPS debounce, geofence, timeout, link loss, absent-battery warning.
Verified mode transition	Tested against a ModeRejectingVehicle mock - a vehicle that accepts the dronekit setter but never adopts the mode. Both the success-after-retry and the give-up paths are asserted.
Abort/landing interlock	test_abort_blocks_until_vehicle_disarms, test_abort_cross_fallback_rtl_to_land, test_preflight_interlock_refuses_armed_vehicle.
Packet security	Round-trip, bit-flip tamper rejection, wrong-key rejection, replay rejection, bad length and bad magic - all in tests/test_hub.py.
Hub decision chain	Pipeline dispatches on a verified scream, does not on a quiet clip, rejects unknown nodes and replays, and degrades correctly with no clip.
Obstacle geometry	7 pure-geometry tests including two-obstacle routing and projection round-trip.
End-to-end autonomous flight	tests/test_full_mission.py in ArduPilot SITL: 5 consecutive passes recorded, one run logged at 331.3 s with 0.6 m closest approach and 8/8 required checks.
Zero-hardware full chain	scripts/demo_phase0.py: synthesised scream -> sealed packet -> hub pipeline -> dispatch -> SITL flight -> kit drop -> RTL.
Phone demo path	5 tests plus a live browser path: real phone-mic audio through the real Stage-1 and Stage-2 code into a dispatch decision.

Not yet done - say these plainly if asked

Gap	The honest answer
No hardware has been built or flown	Everything is validated in simulation and on the phone path. Phases 1-4 in docs/PROJECT_PLAN.md are the hardware plan: audio bench, LoRa range, drone build with mandatory RC override and VLOS, then the integrated field demo.
firmware/node/stage1.cpp has never been compiled on hardware	Its own header says so. The MFCC must be verified against ml/mfcc.py on a shared test clip before it is trusted - that is the first Phase-1 task.
Detection accuracy is unmeasured	The committed model is trained on a bootstrap dataset that validates the pipeline only. The trainer that reports honest metrics (train_gpu.py) exists and is ready; it needs real data.
Stage 2 runs the heuristic on any dev machine	PANNs needs torch, which is only installed on the Pi. The fallback is labelled as such in code, in the logs, and on the dashboard.
dronekit is unmaintained	It needs a collections-ABC shim to import at all on Python 3.10+. Migrating to pymavlink and ArduPilot 4.x SITL is the number-one engineering roadmap item.
Docker path unverified	docker-compose.yml exists and was repaired by review (healthcheck gating, API_UPSTREAM proxy) but has never been run - there is no Docker on the dev machine.
No CI	GitHub Actions to run the unit tier on every push is a known roadmap item.
Patents not filed	Two IPO Form-2 drafts exist. Note the risk out loud: the repository is public, which is self-disclosure, so filing is time-critical.
Payload release has no physical confirmation	The code confirms the servo command was sent, not that the kit left the aircraft. A payload microswitch is named as future work in payload_release.py.
No sensor-based obstacle avoidance	Only map-based routing around configured zones. Reactive avoidance needs a rangefinder or depth camera plus ArduPilot 4.x object avoidance.

Gap	The honest answer
Default keys are development keys	The AES master key is hard-coded as 000102...0e0f in both node.ino and hub/config.py. Deployment must set HUB_MASTER_KEY and provision per installation.
Author fields and plagiarism check	The paper, thesis and patent drafts still carry '(to be filled in)' author placeholders, and Turnitin has not been run.

11 Fifty rehearsed panel questions

Read the answer aloud once. The wording is chosen to be short, concrete, and to volunteer the limitation before the panel has to dig for it.

Concept and motivation

Q. In one sentence, what is your project?

A. A distributed acoustic sensing network that detects human distress on-device at the street pole, verifies it with a second, larger model at a local hub, and automatically dispatches a drone to the verified location with a camera and a first-aid kit - all without internet.

Q. Why not just a mobile app or a panic button?

A. Both require the victim to act - to reach a phone, unlock it, press something. Acoustic detection is passive: a scream is involuntary. The two are complementary, not competing; our /trigger API would accept an app-generated alert unchanged.

Q. Why a drone and not just alerting the police?

A. The hub does log and alert; the drone is what closes the gap between the alert and someone arriving. It reaches the spot at ~15 m/s in a straight line, starts recording evidence immediately, and can deliver a first-aid kit. It is a first responder, not a replacement for one.

Q. What is actually novel? 'Send a drone to a GPS point' exists.

A. Agreed, and our own prior-art scan says exactly that - US10216181B2, US10089889B2 and US12184803B2 cover trigger-to-GPS dispatch. What we claim is the verification and interlock layer: two-stage acoustic verification before any aircraft is committed, and a flight stack where every mode command is confirmed by autopilot telemetry before the mission proceeds. Those are our two patent drafts.

Architecture and code

Q. Which code goes onto which hardware?

A. Two ESP32 boards get code we wrote and burned: firmware/node/ on the sensing node, firmware/gateway/ on the LoRa bridge. The Pi 5 hub runs the hub/ Python package. The companion computer on the drone runs flight_core/ plus trigger_api/. The Pixhawk runs stock ArduPilot - none of our code goes on it; we only send it MAVLink.

Q. Why is your Stage-1 model so small?

A. Because it has to fit on the pole. The deployed model is 748 parameters, about 3 kilobytes of floats, and needs no ML library on the device at all - three matrix multiplies. That is deliberate: Stage 1 only has to decide 'worth a look'. Stage 2 on the Pi does the hard classification.

Q. Why two stages instead of one good model?

A. PANNs/CNN14 is around 80 MB of weights. An ESP32-S3 has about 512 KB of SRAM. There is no one model that fits both constraints. The cascade also gives three free wins: 99.8% less audio transmitted, no continuous audio leaving the pole, and a power budget a solar node can meet.

Q. Why LoRa and not 4G?

A. No SIM, no subscription, no internet, 5-10 km per hop. The areas that most need this are exactly the ones with the least reliable connectivity. 4G at every pole is also a recurring cost per node forever.

Q. How does the hub know where the incident is if the packet is only 25 bytes?

A. The packet carries a node id, not coordinates. Each pole is surveyed once at installation and stored in hub/nodes.json, so the registry supplies the lat/lon. That saves a GPS module per node, its power, and its fix-acquisition delay.

Q. What happens if the WiFi clip never arrives?

A. The system degrades rather than failing. `hub/pipeline.py:93` sets the audio score to the Stage-1 confidence times 0.6, so a dispatch still becomes possible on the LoRa alert alone - but it now needs a Stage-1 confidence of 0.83 to clear the 0.50 verify gate. Fail-degraded, not fail-open.

Q. Where is the dispatch decision actually made?

A. `hub/pipeline.py:104`. Two independent thresholds must both pass: audio score at or above 0.50 and fused severity at or above 0.60. Everything else - authentication, verification, fusion - feeds those two numbers.

Algorithms, formulas and ML

Q. Explain MFCC in your own words.

A. It converts 32,000 raw samples into 123 by 13 numbers describing the shape of the sound spectrum the way human hearing perceives it. Six steps: pre-emphasis to lift the highs, Hamming windowing into 32 ms frames, FFT to get the power spectrum, 40 triangular filters spaced on the mel scale, a logarithm, then a DCT to decorrelate and keep the 13 lowest coefficients.

Q. Why the mel scale specifically?

A. Human pitch perception is roughly logarithmic - we resolve 100 versus 200 Hz easily but not 5000 versus 5100. $\text{mel}(f) = 2595 \log_{10}(1 + f/700)$ matches that, so the filters spend more resolution where the information is.

Q. Why the DCT at the end?

A. Adjacent mel bands are highly correlated. The DCT decorrelates them and concentrates the energy in the first few coefficients, so 13 numbers carry most of what 40 carried. It also shrinks the model input, which matters on a microcontroller.

Q. Why write your own MFCC instead of using librosa?

A. Feature parity. The model is trained in Python and runs in C on the ESP32. librosa's exact filterbank and DCT normalisation are awkward to reproduce in C, and a small mismatch silently degrades a deployed model. One hand-written definition, mirrored step for step in `stage1.cpp`, plus `tests/test_mfcc.py`, makes the parity checkable.

Q. What is your loss function and optimiser?

A. Categorical cross-entropy on softmax outputs, optimised with Adam - learning rate 3e-3, beta1 0.9, beta2 0.999, epsilon 1e-8, with bias correction. Both are hand-implemented in about eight lines of NumPy in `ml/train_stage1_numpy.py`, so I can show you the gradient.

Q. Derive the gradient of softmax with cross-entropy.

A. It collapses to $dL/d\text{logits} = (p - Y)/N$, where p is the softmax output and Y the one-hot label. That cancellation is exactly why the two are always paired, and it is why the code has no separate softmax-derivative term - line 113 of the trainer is that formula.

Q. Why He initialisation?

A. ReLU zeroes roughly half the activations, so variance shrinks layer by layer. He scaling, W drawn from a normal with standard deviation $\sqrt{2/n_{\text{in}}}$, compensates for that. Xavier assumes a symmetric activation like tanh and under-scales for ReLU.

Q. Why mean and std pooling? Doesn't that throw away time?

A. It does, and that is the real limitation - the pooled model cannot distinguish 'help' from the same phonemes reversed. It was chosen because feeding all 1599 features to a dense layer would need 38,000 weights and overfit a small dataset. The std term is doing real work: burstiness over time is what separates a scream from steady traffic. The CNN path in `ml/train_stage1.py` keeps the ordering and is the next step.

Q. What is int8 quantisation and why use it?

A. Weights and activations are stored as 8-bit integers with a per-tensor scale S and zero point Z : $q = \text{clamp}(\text{round}(x/S) + Z, -128, 127)$, and back the other way $x = (q - Z) * S$. The scales are calibrated by running about 200 real samples through the float model. It gives a 4x smaller model and faster integer arithmetic on a microcontroller with no vector FPU.

Q. What is PANNs and why did you not train your own Stage 2?

A. Pretrained Audio Neural Networks - CNN14 trained on Google's AudioSet: 2 million clips across 527 sound classes, including Screaming, Shout, Yell and Crying. That is orders of magnitude more supervision than this project could collect. We sum the probabilities of the distress-relevant classes into one score. Fine-tuning it on local data is the correct next step and would beat both training from scratch and using it off the shelf.

Q. Why sum those class probabilities instead of taking the maximum?

A. Because several AudioSet labels describe the same event and split the probability mass between them. Summing recombines it; $\min(1.0, \dots)$ keeps the result inside $[0, 1]$.

Q. Your fusion weights - where did they come from?

A. They are prototype values, and the docstring in `hub/fusion.py` says so explicitly. They sum to 1.00 and encode a deliberate ordering: Stage-2 audio dominates at 0.60, the node's own confidence adds 0.15, motion 0.10, darkness 0.08, night hours 0.07. Tuning them against Phase-1 bench data is the plan; claiming they are learned would be false.

Q. Why not use a machine-learned fuser?

A. Two reasons. There is no labelled incident data to learn from yet. And every dispatch has to be explainable after the fact - the `Severity.reasons` string carries the full evidence trace into the log and the dashboard. A learned fuser trades that away.

Q. Where is the shortest-path algorithm? I don't see Dijkstra or A*.

A. A quadcopter is not on a road graph, so straight-line distance IS the true path length - that is why nearest-station selection is an `argmin` over haversine distances, in `hub/sim_drone.py:174`. Graph search belongs where there are obstacles, and there we use deterministic geometry to route around configured keep-out zones. If those zones ever got dense, a visibility graph with A*, or RRT*, would replace it.

Q. Why haversine rather than Euclidean distance?

A. A degree of longitude shrinks with $\cos(\text{latitude})$. At Nagpur's 21 degrees north, treating degrees as a flat plane would misjudge distances by about 7% - enough to put the 5 km geofence in the wrong place. Haversine uses the correct spherical formula, and we use the numerically stable `arcsin` form.

Q. Why not Vincenty for full ellipsoidal accuracy?

A. Haversine's spherical error is about 0.3% - roughly 15 m over 5 km. Our waypoint tolerance is 5 m and the geofence is 5000 m, so ellipsoidal precision would be invisible while adding a dependency.

Security

Q. How do you stop someone spoofing an alert and launching your drone?

A. Every packet carries an HMAC-SHA256 tag computed with a key derived per node from a master key. The hub verifies that tag with a constant-time comparison BEFORE it decrypts anything, and rejects any counter that is not strictly greater than the last one seen from that node. Tamper, wrong-key and replay rejection are all unit-tested in `tests/test_hub.py`.

Q. Why CTR mode and not CBC?

A. CBC needs padding, which would grow an 8-byte payload to 16 and make the radio packet 47% larger. CTR is a stream mode: ciphertext length equals plaintext length, no padding. And ECB is never acceptable - identical plaintext would produce identical ciphertext.

Q. What is the biggest risk in your crypto design?

A. Nonce reuse. CTR builds its keystream from (key, nonce); reusing a pair lets an attacker XOR two ciphertexts and cancel the keystream entirely. That is exactly why the counter is stored in the ESP32's NVS - if it reset to zero on reboot, we would both break replay protection and reuse nonces on a solar node that power-cycles.

Q. Why truncate the HMAC to 64 bits?

A. LoRa airtime. A forger gets one chance in 2^{64} per attempt, and at SF9 data rates brute force is not practical. It is an explicit trade, documented in the file header.

Q. Would you change anything about the crypto?

A. Yes - AES-GCM or AES-CCM would give authenticated encryption in a single standard pass, and CCM is what LoRaWAN itself uses. We hand-composed CTR plus HMAC because mbedtls on the ESP32 and PyCryptodome on the Pi both expose exactly those primitives, which made the two 40-line implementations auditable and provably identical. And the default master key in the repo is a development key - a real deployment must provision HUB_MASTER_KEY per site.

Q. Is the gateway a security weak point?

A. No, deliberately. It holds no keys and does no verification - it just prints received bytes as hex on serial. Stealing it gains an attacker nothing they could not get with their own radio.

Flight safety

Q. What is the single most important safety mechanism?

A. The verified mode transition. Every flight-mode change is a request, not a fact: it is re-issued as raw MAVLink every 700 ms until the autopilot's own heartbeat reports the new mode. It exists because we found ArduCopter 3.3 silently ignoring dronekit's mode setter - the client believed GUIDED while the autopilot stayed in STABILIZE.

Q. What if two failsafes fire at once?

A. There is an explicit arbitration lattice in failsafe_handler.py:_emit. LAND outranks RTL and is never downgraded, so a 19% low-battery RTL can never overwrite a 9% critical-battery LAND. And the RTL loop re-checks the arbiter every second, so it will switch to LAND mid-return if the demand escalates.

Q. Why does GPS loss trigger LAND and not RTL?

A. Because with no position fix, RTL cannot navigate - it has no idea which way home is. Landing in place is the only safe autonomous response. And it is debounced over three consecutive bad samples, so one noisy reading never puts the aircraft down.

Q. Can two missions overlap?

A. No, structurally. There is one aircraft, so the queue is serial. Every abort path blocks until the vehicle has landed AND disarmed, and run_mission independently refuses to start while the vehicle is armed. The invariant is that the queue can never fly against an airborne vehicle.

Q. What if the operator wants to recall the drone?

A. POST /mission/{id}/cancel, or the button on the dashboard. A queued mission is removed immediately; a running one aborts to RTL, and the queue stays blocked until it is safely down.

Q. You disabled the pre-arm checks. Isn't that dangerous?

A. Only in simulation, and only when SITL_MODE=1. ArduCopter 3.3 refuses to arm without an RC transmitter, which SITL has none of. On real hardware that environment variable is unset and the function logs 'real-hardware mode: leaving ArduPilot pre-arm checks at stock values' without touching a single parameter. Separately, every real flight requires an RC transmitter with a mode switch as the human kill path.

Q. What if the kit fails to drop?

A. The mission reports the failure and continues to RTL. The design rule is written into payload_release.py: a failed release is never a reason to loiter. Note also that we confirm the servo command was sent, not that the kit physically left - that needs a payload microswitch, which is named as future work.

Q. Do you have obstacle avoidance?

A. Map-based, yes: deterministic geometric routing around operator-configured cylindrical keep-out zones, with 7 unit tests. Sensor-based reactive avoidance, no - nothing in the repo reads a rangefinder or depth camera. That needs extra hardware plus ArduPilot 4.x's own object avoidance, and it is on the roadmap. The module docstring states this in its first paragraph.

Testing, results and the road ahead

Q. How do you know any of this works?

A. 80 unit tests pass in 21 seconds - I ran them today - plus an end-to-end autonomous flight in ArduPilot SITL that has passed five consecutive times, with one run logged at 331 seconds, 0.6 m closest approach to the target and 8 of 8 required checks. On top of that, scripts/demo_phase0.py runs the entire chain from a synthesised scream to a completed flight with zero hardware.

Q. How do you test a safety mechanism you cannot reproduce in a simulator?

A. With a mock. tests/test_units.py defines a ModeRejectingVehicle - a vehicle that accepts the dronekit setter but never actually adopts the mode. That is precisely the failure we found in the real autopilot, and it cannot be triggered on demand in SITL. The mock lets us assert both the retry-until-confirmed path and the give-up path in milliseconds.

Q. What is your detection accuracy?

A. I will not quote one, because the committed model was trained on a bootstrap dataset that exists to validate the pipeline, not to measure detection. Quoting that number would be misleading. The measurement harness is already written - ml/train_gpu.py reports per-class precision, recall, F1, a confusion matrix and the background false-alarm rate, and it splits by file to avoid leakage. Producing those numbers on real recordings is the Phase-1 deliverable.

Q. What is your next milestone?

A. Phase 1, the audio bench: ESP32-S3 plus INMP441 capturing real audio, the trained Stage-1 model flashed, PANNs running on the Pi 5, and measured detection distance, latency and false-positive rate. The first task inside it is verifying that the C MFCC matches the Python MFCC on a shared test clip, because everything downstream depends on that.

Q. What is the biggest technical debt?

A. dronekit. It is unmaintained - it needs a collections-ABC shim just to import on Python 3.10+, and it pins us to the copter-3.3 simulator. Migrating to pymavlink directly with ArduPilot 4.x SITL is roadmap item number one, and it would also unlock the autopilot's own object avoidance.

Q. What would you do differently if you started again?

A. Three things. Start the flight stack on pymavlink instead of dronekit. Use AES-CCM instead of hand-composed CTR plus HMAC. And collect real audio before building the model pipeline, so the architecture was chosen against measured data rather than validated afterwards.

Q. How does this scale to a city?

A. The node is the unit of scale, at roughly 2,000-3,000 rupees each. One hub covers every node inside LoRa range - 5 to 10 km - and the registry is just a JSON map of node id to coordinates. The current single-drone queue is the real bottleneck, which is why the fleet abstraction with nearest-station dispatch already exists in hub/sim_drone.py: adding a station is one line in the DRONE_BASES config.

Q. What about privacy? You have microphones on public poles.

A. It is the reason Stage 1 runs on the node. Nothing is streamed - audio stays in a 4-second circular buffer in RAM and is overwritten continuously. Only when a detection fires does a single 4-second clip leave the pole, and it goes to a local hub, not the cloud. So under 0.2% of audio is ever transmitted, and none of it leaves the site. docs/PROJECT_PLAN.md section 7 covers the privacy, legal and DGCA position.

12 Live demo runbook and closing notes

If you get to demo something

Ranked by how impressive they are versus how likely they are to fail on the day. Note: this working copy has no .venv - the commands below assume a Python with the requirements installed. Test whichever one you plan to run BEFORE the seminar.

Demo	Command	Time	What it proves	Risk
Unit test suite	<code>python -m pytest</code>	~21 s	80 tests green: failsafe arbitration, verified mode setter, abort interlock, packet crypto, hub pipeline, MFCC, obstacle geometry	Lowest. Run this one.
Phone / no-hardware demo	<code>python -m hub.main --web-only --https</code> then open <code>https://<pc-ip>:8990/node</code> on a phone	~1 min	Real phone-mic audio through the real Stage-1 and Stage-2 code into a real dispatch decision, with a drone animating on the map	Low, but needs both devices on the same WiFi and accepting the self-signed certificate.
Full chain in SITL	<code>python scripts/demo_phase0.py</code>	~6 min	Synthesised scream -> sealed packet -> hub authenticate/verify/fuse/decide -> dispatch -> real autonomous flight -> kit drop -> RTL	Medium - spawns SITL plus uavicorn as children. Start it before you begin speaking.
Live dashboard	<code>.\run_nagpur.ps1</code> then open <code>http://localhost:5173</code>	~2 min to boot	The operator view: map centred on Nagpur, dispatch button, live breadcrumb trail of a real SITL flight	Medium - three windows, needs npm install in dashboard/ to have been done.
End-to-end flight test	<code>python tests/test_full_mission.py</code>	~5-6 min	Prints PASS/FAIL against 8 required checks	Medium. Run it in the background and show the result, do not watch it live.

Three things to say in your first ninety seconds

1. The problem and the gap: distress is detected late, and the time between 'something happened' and 'someone arrived' is where harm occurs.
2. The architecture in one breath: tiny model on the pole, big model at the hub, encrypted LoRa in between, autonomous drone at the end - and nothing depends on the internet.
3. The honest status: the full chain is validated in simulation with 80 passing tests and a repeatable end-to-end autonomous flight; hardware bring-up and measured detection metrics are Phase 1.

Three things never to say

- Any accuracy percentage from the bootstrap dataset.
- That the drone avoids obstacles using sensors. It routes around a configured map.
- That the system has flown on real hardware. It has flown in ArduPilot SITL, repeatably - which is a real and defensible claim on its own.

Where the documents are, if the panel asks for reading

Document	What it covers
<code>docs/PROJECT_PLAN.md</code>	The v2 master plan: concept, architecture, the four hardware phases, BOM, safety, privacy, legal
<code>docs/SYSTEM_DOCUMENTATION.md</code>	Operator and developer reference for the flight stack: API, config, failsafes, troubleshooting
<code>docs/HARDWARE_INTEGRATION.md</code>	Every pinout, ArduPilot parameter, calibration step, and the SITL-to-hardware switchover
<code>docs/BUILD_AND_OPERATIONS_GUIDE.md</code>	Shopping list (roughly 36,000 rupees minimum BOM), assembly, operations

Document	What it covers
docs/DATASET_AND_TRAINING.md	Classes, data sources, preprocessing, augmentation, splitting, and the metrics to report
docs/HARDWARE_PHASES.md	Phase 1-4 bring-up plan with what to measure at each step
docs/RESEARCH_PAPER.md / THESIS.md	The pre-print and the nine-chapter thesis on the flight stack (both still accurate - the flight core is unchanged in v2)
docs/patents/	Two IPO Form-2 drafts (verified dispatch; failsafe arbiter), the prior-art landscape, and a filing checklist
docs/PHONE_TEST.md	How to run the no-hardware phone demo
CLAUDE.md / AGENTS.md	The orientation document: what exists, what decisions were made and why, and the gotchas learned the hard way

LAST THING

The strongest position in a viva is not 'everything works'. It is 'here is exactly what is proven, here is exactly what is not, and here is the test that will settle it'. This project genuinely is in that position - 80 passing tests, a repeatable autonomous flight in simulation, a security layer with tamper and replay tests, and a clearly stated hardware phase plan. Lead with the proof, name the gaps yourself, and the panel has nothing left to catch you on.